

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ  
ВЫСШЕГО ОБРАЗОВАНИЯ  
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра Систем сбора и обработки данных  
(полное название кафедры)

Утверждаю

Зав. кафедрой Бакаев М.А.

\_\_\_\_\_  
(подпись, инициалы, фамилия)

«\_\_» \_\_\_\_\_ 2025 г.

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА**

Повелицина Станислава Максимовича

(фамилия, имя, отчество студента – автора работы)

Разработка системы автоматизированного хранения и нормоконтроля допуска

(тема работы)

выпускных квалификационных работ

Автоматики и вычислительной техники

(полное название факультета)

Направление подготовки 09.03.02 Информационные системы и технологии  
(код и наименование направления подготовки бакалавра)

в промышленности и бизнесе

**Руководитель  
от НГТУ**

Тетерин Максим Михайлович

(фамилия, имя, отчество)

Старший преподаватель ССОД

(ученая степень, ученое звание)

\_\_\_\_\_  
(подпись, дата)

**Автор выпускной  
квалификационной работы**

Повелицин Станислав Максимович

(фамилия, имя, отчество)

АВТФ, гр. АТ-13

(факультет, группа)

\_\_\_\_\_  
(подпись, дата)

Новосибирск 2025

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ  
ВЫСШЕГО ОБРАЗОВАНИЯ  
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра \_\_\_\_\_ *Систем сбора и обработки данных* \_\_\_\_\_  
(полное название кафедры)

Утверждаю

Зав. кафедрой \_\_\_\_\_

\_\_\_\_\_  
(подпись, инициалы, фамилия)

« \_\_\_\_ » \_\_\_\_\_ 2025 г.

**ЗАДАНИЕ  
НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ БАКАЛАВРА**

Студенту \_\_\_\_\_ *Повелищину Станиславу Максимовичу* \_\_\_\_\_  
(фамилия, имя, отчество)

Направление подготовки \_\_\_\_\_ *09.03.02 Информационные системы и технологии в* \_\_\_\_\_  
(код и наименование направления подготовки бакалавра)  
*промышленности и бизнесе* \_\_\_\_\_  
*Автоматика и вычислительная техника* \_\_\_\_\_  
(полное название факультета)

Тема \_\_\_\_\_ *Разработка системы автоматизированного хранения и нормоконтроля* \_\_\_\_\_  
(полное название темы выпускной квалификационной работы бакалавра)  
*допуска выпускных квалификационных работ* \_\_\_\_\_

Исходные данные (или цель работы) \_\_\_\_\_ *Создание системы для организованного* \_\_\_\_\_  
*хранения, сопровождения и демонстрации ВКР студентов* \_\_\_\_\_

Структурные части работы \_\_\_\_\_

*1. Введение* \_\_\_\_\_

*2. Обзор предметной области* \_\_\_\_\_

*3. Проектирование* \_\_\_\_\_

*4. Реализация* \_\_\_\_\_

*5. Заключение* \_\_\_\_\_

*6. Список литературы* \_\_\_\_\_

[illegible]

(подпись, дата)

(подпись, дата)

изменена приказом по НГТУ № \_\_\_\_\_ от « \_\_\_\_\_ » \_\_\_\_\_ 2025 г.

(фамилия, имя, отчество секретаря государственной

## АННОТАЦИЯ

Выпускная квалификационная работа выполнена студентом Повелицином С.М.

Руководитель работы: Тетерин М.М.

Тема: «Разработка системы автоматизированного хранения и нормоконтроля допуска выпускных квалификационных работ»

Пояснительная записка содержит 74 страниц, 14 рисунков, 1 таблицу и 2 приложения

Ключевые слова: базы данных, СУБД, веб-приложение, JavaScript, Node.js

Целью работы является разработка системы для хранения, сопровождения и контроля выпускных квалификационных работ студентов кафедры ССОД НГТУ.

Перечень используемых сокращений:

- БД – база данных;
- СУБД – система управления базами данных;
- ВКР – выпускная квалификационная работа.

## СОДЕРЖАНИЕ

|                                           |    |
|-------------------------------------------|----|
| Введение.....                             | 5  |
| 1. Обзор предметной области .....         | 7  |
| 1.1 Требования к системе.....             | 7  |
| 1.2 Постановка задачи.....                | 9  |
| 1.3 Обзор существующих решений .....      | 10 |
| 1.4 Подготовка к проектированию .....     | 13 |
| 1.4.1 База данных .....                   | 13 |
| 1.4.2 Выбор языка программирования.....   | 15 |
| 2. Проектирование .....                   | 17 |
| 2.1 Проектирование базы данных.....       | 17 |
| 2.1.1 Набор сущностей с атрибутами.....   | 17 |
| 2.1.2 Набор связей сущностей .....        | 20 |
| 2.1.3 Диаграммы «сущность-связь» .....    | 21 |
| 2.1.4 Наборы отношений .....              | 23 |
| 2.1.5 Нормализация отношений.....         | 25 |
| 2.2 Проектирование веб-приложения.....    | 28 |
| 2.2.1 стек разработки.....                | 29 |
| 2.2.2 Пользовательский интерфейс .....    | 29 |
| 2.2.3 Взаимодействие с базой данных ..... | 30 |
| 2.2.4 Скачивание и загрузка файлов .....  | 30 |
| 2.2.5 Роли пользователей.....             | 31 |
| 2.2.6 Асинхронность .....                 | 34 |
| 2.2.7 Авторизация.....                    | 34 |
| 2.2.8 Почтовая служба .....               | 34 |

|                                                                                        |    |
|----------------------------------------------------------------------------------------|----|
| 2.2.9 Обсуждения .....                                                                 | 35 |
| 2.2.10 Проверки.....                                                                   | 35 |
| 2.2.11 Навигация .....                                                                 | 35 |
| 3. Реализация .....                                                                    | 37 |
| 3.1. База данных .....                                                                 | 37 |
| 3.2. Вспомогательные функции .....                                                     | 38 |
| 3.3. Ключевые алгоритмы.....                                                           | 38 |
| 3.3.1. Авторизация пользователей.....                                                  | 38 |
| 3.3.2. Создание репозитория ВКР.....                                                   | 40 |
| 3.3.3. Загрузка файлов в репозиторий ВКР .....                                         | 42 |
| 3.3.4. Создание обсуждений.....                                                        | 43 |
| 3.3.5. Запрос на проверку работы .....                                                 | 44 |
| 3.3.6. Проверка работы научным руководителем.....                                      | 45 |
| 3.3.7. Проверка работы нормаконтролем .....                                            | 47 |
| Заключение .....                                                                       | 49 |
| Список литературы .....                                                                | 51 |
| Приложение .....                                                                       | 53 |
| Приложение А - Пожелания к системе автоматизированного хранения<br>научных работ ..... | 53 |
| Приложение Б - Листинг кода приложения.....                                            | 57 |

## ВВЕДЕНИЕ

Ежегодно кафедра ССОД НГТУ выпускает десятки студентов, немалую часть проектов которых составляет разработка программного кода и скриптов для различных задач или приложений. Это могут быть мобильные приложения, автотесты, сайты или программное обеспечение для уникального оборудования.

Однако после защиты большинство таких проектов теряют практическую значимость. Студенты загружают электронные версии пояснительных записок в личный кабинет и редко возвращаются к ним, а само программное обеспечение в лучшем случае остаётся в папке на компьютере научного руководителя. Бумажные копии документов отправляются в архив, где хранятся несколько лет перед утилизацией. Таким образом, результаты труда, которые могли бы представлять ценность для кафедры, потенциальных работодателей или будущих студентов, фактически исчезают из информационного поля.

Проблема усугубляется отсутствием систематизированного доступа к этим работам. Если кому-то потребуется узнать, каких специалистов может предложить кафедра, придётся либо обращаться к сотрудникам лично, либо вручную изучать сотни выпускных работ на портале НГТУ или в архиве. Такой подход крайне неэффективен: он требует значительных временных затрат, а потому редко применяется на практике. Как следствие, потенциал многих проектов остаётся нераскрытым, а связь между академической подготовкой и реальными потребностями рынка — слабо прослеживаемой.

Решением могла бы стать специализированная система для хранения выпускных работ. Такой инструмент не только обеспечил бы централизованное хранение программного обеспечения и сопутствующей документации, но и позволил бы:

- Упростить поиск проектов по тематике, технологиям или ключевым словам.
- Сохранить преемственность между поколениями студентов, избегая дублирования уже решённых задач.
- Повысить прозрачность компетенций кафедры для внешних заинтересованных сторон (например, работодателей).

Актуальность подобной системы подтверждается и мировым опытом. Многие университеты, включая НГТУ, внедряют репозитории для студенческих работ (например, на базе GitHub, GitLab или внутренних платформ), что упрощает интеграцию выпускников в профессиональную среду. Для кафедры ССОД НГТУ такой подход мог бы стать шагом к цифровой трансформации образовательных процессов.

Цель данной работы — создание системы для организованного хранения, сопровождения и демонстрации ВКР студентов. Для чего необходимо определить требований к функционалу, проанализировать существующие аналоги, спроектировать структуры данных и интерфейса, а также определить возможности интеграции данной системы, с уже имеющейся в НГТУ информационной архитектурой.



# 1. ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ

## 1.1 Требования к системе

Основная цель создаваемого программного продукта - хранение разрабатываемых в рамках выпускной квалификационной работы проектов студентов и документов. Для этого должно быть разработано хранилище, а также возможность загружать и скачивать данные проекты и документы из него. Для удобства пользования, лучше всего предоставить студентам возможность самим загружать их проекты, т.к. это снизит нагрузку на сотрудников кафедры. Для обеспечения безопасности системы, необходимо, чтобы пользователи проходили аутентификацию и авторизацию. Преимуществом будет, если в системе будут иметься различные пользователи с разными правами: студенты, научные руководители, администраторы и заведующий кафедрой, и т.д.

Так же на начальном этапе работы были опрошены некоторые сотрудники кафедры (преподаватели, нормаконтроль и заведующий кафедрой), а также студенты на предмет того, что они хотели бы видеть в разрабатываемой системе. Ниже представлена часть пожеланий в виде “пользовательских историй”. Полный список пожеланий представлен в приложении А.

Пожелания заведующего кафедрой:

- Хотелось бы, чтобы высылались оповещения на почту о том, что что-то надо загрузить или о том, что что-то было загружено, чтобы постоянно не проверять через сам репозиторий;
- Хотелось бы иметь счетчик студентов, загрузивших работу в целом, а также у конкретного научного руководителя, чтобы знать прогресс у студентов, а также знать у какого научного руководителя отстаёт прогресс;

- Хотелось бы иметь возможность добавить работу в избранное, чтобы было проще найти её в будущем.

#### Пожелания норма контроль:

- Хотел бы иметь возможность ставить отметку на пояснительной записке, чтобы можно было отправить её на доработку;
- Хотелось бы иметь возможность оставлять замечания для всей пояснительной записке, чтобы студенты знали, что не так с их работой;
- Хотелось бы иметь возможность диалога со студентом, чтобы иметь обратную связь по замечаниям к работе;
- Хотелось бы иметь чек-лист, список которого бы содержал необходимые критерии, которые должна пройти пояснительная записка во время проверки у норма-контролёра, чтобы эти критерии были перед глазами во время проверки.

#### Пожелания научных руководителей:

- Хотелось бы иметь возможность загружать ВКР, чтобы это могли делать не только студенты или ответственные за это лица.
- Хотелось бы, чтобы система отправляла на почту оповещения, чтобы постоянно не проверять систему;
- Хотелось бы, чтобы у выпускных работ отображались оценки, полученные за их защиту, чтобы не искать эти оценки в других источниках;
- Хотелось бы, чтобы студентам приходили оповещения об сроках сдачи контроля работ, чтобы не оповещать их самолично или они не искали, где это можно узнать;
- Хотелось бы, чтобы отображались замечания с защиты чтобы знать, о замечаниях, если не присутствовал во время неё.

#### Пожелания студентов:

- Хотелось бы, чтобы в системе имелась возможность провести ревью выпускной работы, чтобы научный руководитель (или другой

проверяющий) смог оценить изменения перед загрузкой работы в репозиторий.

Изучив пожелания, были сделаны выводы, что большой интерес представляет обратная связь по ходу работы: студенты хотели бы, чтобы научные руководители могли проводить ревью работ, научные руководители хотели бы видеть различные оценки работ, нормаконтроль хотел бы ставить замечания к работам и вести диалог со студентами для обсуждения того, что у них не верно. Так же научные руководители хотели бы, чтобы в системе отображались сроки различных проверок и этапов, которые должны пройти работы студенты, а также предварительные оценки за работы по итогам этих этапов. Ещё, чтобы система могла оповещать пользователей по на почту об различных событиях внутри неё. Т.е., чтобы эта система могла ещё использоваться для сопровождения ВКР, а не только хранить результаты работ.

## **1.2 Постановка задачи**

Требуется разработать отдельную систему, которая занималась бы хранением ВКР студентов, а также помогала в их сопровождении в момент разработки.

Система должна уметь хранить не только пояснительные записки, но и сами проекты студентов, которые они разрабатывают в рамках выпускных работ, если имеется такая возможность и необходимость, например, если проектом является разработка ПО. Должны также храниться оценки работ студентов.

Студенты и научные руководители должны иметь возможность, собственно, загружать и обновлять эти работы.

Ещё необходимо, чтобы на этапе преддипломной практики студенты могли использовать систему для предварительной демонстрации научным руководителям и нормаконтролю для оценки и корректировки того, что они

делают, с помощью обсуждений работ, написанием заметок или задач, которые эти студенты должны были бы выполнить для них.

### **1.3 Обзор существующих решений**

В информационной системе НГТУ уже имеется возможность хранения ВКР студентов с итоговой оценкой, а также от антиплагиата.

**1. ВКР-СМАРТ/ВКР-ВУЗ.РФ[1]** – это система, разработанная компанией «Профобразование». Данная система включает в себя возможность хранения ВКР, а также их проверки на заимствование и антиплагиат.

В системе имеются различные группы пользователей: сотрудник, администратор, преподаватель, студент, проверяющий, со своими привилегиями и возможностями для управления работами.

Загружать могут администраторы (представители организаций), сотрудники подразделений и обучающиеся (работы которых модерируют сотрудники).

- Администраторы могут регистрировать новых пользователей, назначать роли, создавать архивы ВКР, загружать или удалять их, создавать подразделения ВУЗа.
- Сотрудники подразделений могут загружать работы и проводить модерацию работ студентов, которые они загружают.
- Обучающиеся могут загружать работы.

Имеется система модерации работы с принятием или отклонением работ для последующей доработки.

Хранения производится для каждого вуза отдельно на виртуальном или реальном сервере.

Автоматическая проверка на плагиат и заимствования при загрузке работы, создание отчетов по плагиату и заимствованиям.

Имеется режимы поиска заимствований в двух вариантах: в автоматическом после загрузки работ и в ручном, а также вариант «Не проверять работу после загрузки». Все режимы выбираются перед началом загрузки работ.

Работы можно загружать пачками или по отдельности.

Имеются свои базы данных для проверки работ на заимствования:

1. База открытых источников в Интернете: постоянно пополняемая база текстов, находящихся в свободном доступе в сети Интернет (базы рефератов, курсовых, Википедия, авторефераты, периодика и т.п.), которые являются наиболее популярными среди студентов для получения текстов к работам. Данной проверки в большинстве случаев хватает для оценки оригинальности работы.

2. Лицензионная база научной и учебной литературы с авторскими правами из более чем 155 000 изданий ЦОР IPR SMART.

3. Едина база работ учебного заведения – индексация загруженных в систему работ, позволяет проверять последующие работы по архиву учебного заведения. Может быть отключена.

4. Индексная база всех работ пользователей системы – все работы, загруженные в систему, на условиях договора включаются в индексную базу системы.

Так же в этой системе имеется электронное портфолио достижений.

Несмотря на обширную базу текстовой документации дипломных работ и модули анализа на заимствования, данная система способна хранить и контролировать только текстовые документы, но не программное обеспечение, что повторяет уже имеющийся функционал в электронной системе НГТУ, и смысла в ней из-за этого нет.

Больше подобных систем найдено не было, что может означать непопулярность данной проблемы.

Другим интересным решением является система контроля версий GIT.

**2. Технология GIT** – это система контроля версий программного продукта, которая сейчас активно используется по всему миру. Изначально она разрабатывалась для того, чтобы несколько разработчиков могли вести работу над одним кодом вместе. Эта технология позволяет управлять сразу несколькими версиями продукта одновременно, а также объединять их и отслеживать изменения. Всё это происходит благодаря репозиториям – хранилищам проектов.

Для работы с репозиториями git имеются специализированные сайты, наиболее популярные из которых – это Github и GitLab.

Github имеет у себя возможность размещения сайтов, но за определенную плату, с другой стороны Gitlab имеет как официальный веб-сайт, так и распространяемую версию для установки на локальных серверах для создания своего хранилища репозитория.

Среди распространяемых версий имеется подписочная (Enterprise Edition или EE) и бесплатная открытая (Community Edition или CE) версии. В состав EE версии входят тот же функционал, что и в CE версии, но он имеет дополнения.

Для загрузок репозитория в GitLab используются 2 протокола HTTP/HTTPS и SSH. При использовании первого протокола для каждого действия необходимо подтверждения при помощи логина и пароля, а при использовании второго необходимо в GitLab добавить специальный SSH-ключ.

GitLab для управлением содержимым использует так называемые “проекты”. В этих проектах хранятся репозитории с различными ветками (версиями) проектов. Система имеет возможность настройки доступа к этим проектам: проектом могут управлять несколько человек и каждому из них могут выдаваться различные привилегии, например, только скачивать содержимое, управлять загрузкой в проект, сливать ветки и т.д.

Данные проекты можно объединять в группы. Так же в состав этих групп могут входить множество пользователей, что упрощают работу в команде над

какой-то системой из множества компонентов. Группам можно настраивать иерархию, видимость для других пользователей и членов групп, активности: сливание и управление ветками, выдавать задачи.

Локально установленный GitLab имеет по умолчанию одного пользователя – администратора. Он может регистрировать новых пользователей, создавать и менять их профили, делать тоже самое с группами и проектами. Выдавать привилегии и ограничивать доступ к проектам из вне.

Git-репозитории позволяют вести управления над любыми проектами и файлами, т.е. можно вести учет изменений и версий не только ПО, но и документации, хотя имеются тут и свои проблемы.

Но GitLab слабо поддаётся самостоятельной модификации из-за чего его будет сложно или невозможно привести к виду, необходимому для решения данной проблемы.

Поэтому систему придётся делать самостоятельно.

## **1.4 Подготовка к проектированию**

В данной главе описывается процесс проектирования базы данных и веб-приложения репозитория, с учетом некоторых требований, собранных перед началом работы. Рассматривается набор сущностей для БД и процесс их приведения к таблицам, выбор стека разработки и

### **1.4.1 База данных**

База данных – это самая важная часть будущей системы, поэтому к её проектированию надо уделить особое внимание, т.к. она будет отвечать за хранение всех ВКР. Существует множество видов проектирования баз данных, но текущее будет сделано с помощью модели «сущность - связь», которая предназначена для логического представления данных.

Модель «сущность - связь», разработанная Питером Ченом, основана на определении наиболее важных данных, называемых *сущностями* и

установления *связи* между ними. Для этих сущностей и связей так же определяются *атрибуты, ключи*.

- Сущность – это отдельный тип объекта, который необходимо представить в базе данных (человек, вещь, место и т.д.). Они бывают реальные (человек, автомобиль) и концептуальные (проект организации, план строительства дома) [2].

Сущности бывают слабые и сильные. Сильная или независимая сущность – это сущность, которая не зависит от других. Слабая или зависимая в свою же очередь – это сущность, существование которой зависит от сущности другого типа

- Атрибуты – это какая-то характеристика сущности, служащая для её идентификации (цвет, масса, дата изготовления и т.д., если сущностью служит автомобиль). Иногда идентификация, может быть, одинаковая для нескольких сущностей (например цвет для автомобиля и для футболки).

Атрибуты бывают простые и составные. Простые атрибуты состоят из одного компонента. Например, фамилия врача. Составные атрибуты – это атрибуты, которые состоят из нескольких компонентов, каждый из которых независим. Например, ФИО – составной атрибут, но фамилия, имя, отчество – простые. [2]

- Ключ – это атрибут или минимальных их набор, значения которых будет однозначно идентифицировать каждый экземпляр типа сущности. [2,3]. Бывают первичные и внешние ключи.

*Первичный ключ* – это ключ, который был выбран для того, чтоб можно было однозначно идентифицировать каждый экземпляр сущности.

*Внешний ключ* – это ключ, который используется при связи некоторой сущности (сущность А) с другой (сущность Б). Если сущность А связана с сущностью Б, то сущность А должна включать в себя внешний ключ, равный первичному ключу сущности Б.

- Связь – это ассоциирование двух или более сущностей. У связи бывает степень, которая показывает количество типов сущностей



участвующих в связи. Наиболее распространённой является бинарная связь (связь между двумя типами сущностей).

У бинарных связей выделяют так же кратность – это количество возможных экземпляров сущности одного типа, связанное с количеством возможных экземпляров сущностей другого типа. [2]

Потом модель «сущность - связь» будет трансформирована в реляционную, потому что в рамках учебной программы имелся опыт подобного проектирования баз данных, а ещё реляционные модели являются наиболее популярным видом БД. Для их написания используется SQL - Structured Query Language или язык структурированных запросов. Из-за давности данного языка (первая статья появилась в 1974-ом году [4]) он имеет множество стандартов, которые продолжают обновляться под нужды пользователей, а также уже большое количество готовых решений под различные нужды, включая как и создание эффективных БД («MySQL по максимуму» [5] - Бэрон Шварц), так и обращения к ней самой («SQL Сборник рецептов» [6] Энтони Молинаро и Робер де Граафа).

#### 1.4.2 Выбор языка программирования

Разрабатываемое приложение должно иметь две части: пользовательскую (или frontend) и логическую/серверную (или back-end) части, без которой оно не сможет в полной мере быть удобной для пользователей, а также удовлетворять их желания.

Frontend отвечает за пользовательские функции и интерфейс. Сюда относится всё, что пользователь будет видеть на экране, при посещении сайта: текст, кнопки, меню, навигация и т.д. Без проработки этой части, людям будет сложнее воспринимать информацию и неудобно использовать функционал, который им предоставляет разработчик. Для этой части почти всегда используется связка из нескольких языков: HTML для указания сайту как отрисовывать страницу, CSS для её стилизации и JavaScript для запросов к серверу и создания интерактивных частей, который был создан как раз для этой фронтенда.

Серверная часть или backend отвечает за то, что будет делать система. Сюда относится аутентификация и авторизация, обработка запросов пользователей, взаимодействие с базой данных. Для этой части используется гораздо больше языков, наиболее популярные из которых это Python, Java, C# [7], но также используется и JavaScript с появлением Node.js, превращающий JavaScript в язык общего пользования, вместо языка для браузера, каким он изначально был.

Чтобы упростить разработку, для клиентской части будет использоваться JavaScript, а для серверной – фреймворк Express для платформы Node.js. О том, как использовать Node.js и Express в веб-разработке, имеется не мало источников, включая интернет видео, статьи, а также книги (например «Веб-разработка с применением Node и Express» [8] или «Node.js. Разработка серверных веб-приложений на JavaScript» [9]).

На основе выбранной модели данных, далее будет спроектирована база данных для будущего решения, а используя выбранный язык программирования, создана система, которая должна решить поставленные задачи.

## **2. ПРОЕКТИРОВАНИЕ**

В данной главе описывается процесс проектирования базы данных и веб-приложения репозитория, с учетом некоторых требований, собранных перед началом работы. Рассматривается набор сущностей для БД и процесс их приведения к таблицам, выбор стека разработки и

### **2.1 Проектирование базы данных**

Проектирование баз данных является критически важным этапом разработки информационных систем, так как от его качества напрямую зависит эффективность хранения, обработки и извлечения данных. Грамотно спроектированная база данных обеспечивает целостность информации, минимизирует избыточность и противоречивость данных, а также оптимизирует производительность системы. Кроме того, хорошо продуманная структура упрощает масштабируемость и поддержку системы в будущем, снижая затраты на доработки. Таким образом, тщательное проектирование базы данных — это фундамент для создания надежной, безопасной и высокопроизводительной информационной системы.

#### **2.1.1 Набор сущностей с атрибутами**

База данных нужна для того, чтобы хранить всю необходимую информацию: пользователей, ВКРы студентов, их оценки и проекты, и т.д. Поэтому важность её правильной проектировки очень велика. Поскольку для проектирования используется модель «сущность - связь», то сначала будут описаны сущности будущей БД, а затем определены связи между ними.

Для части БД, относящейся к пользователям, требуются сущности, отвечающие за хранение информации о них: логины, пароли, права в системе и т.д. Для хранения общей информации пользователя будет использоваться сущность соответственно называемая “пользователь”, определять роли

пользователей в системе будет сущность “роль”. Из которой можно будет выбрать, кем будет пользователь в системе: студентом, научным руководителем, администратором и т.д. Для организации учебных групп сущность “группа”, которая будет хранить названия этих групп. Для связи сущностей “пользователь” и “роль” будет использоваться “роль пользователя”. Для хранения информации об пользователях с ролью “научный руководитель” будет использоваться сущность “руководитель”.

- Сущность – **Пользователь**. Атрибуты: Код\_пользователя (является первичным ключом), Логин\_пользователя, Пароль\_пользователя, Имя\_пользователя, Фамилия\_пользователя, Отчество\_пользователя, ВремяДата\_создания, ВремяДата\_редактирования.

- Сущность – **Роль**. Атрибуты: Код\_Роли (является первичным ключом), Название\_роли.

- Сущность – **Роль\_пользователя**. Атрибуты: Код\_пользователя (является частью первичного ключа и внешним ключом), Код\_роли (является частью первичного ключа и внешним ключом).

- Сущность – **Группа**. Атрибуты: Код\_группы (является первичным ключом), Название\_группы, ДатаВремя\_создания.

- Сущность – **Студент**. Атрибуты: Код\_пользователя (является первичным ключом и внешним ключом), Код\_группы (является внешним ключом).

- Сущность – **Руководитель**. Атрибуты: Код\_пользователя (является первичным ключом и внешним ключом), Звание, Должность.

Несколько сущностей будет отвечать за информацию, связанную с работами. Главная сущность “диплом” будет отвечать за хранение основной информации: кто сделал, тема работы, научный руководитель, оценка. Для хранения документов, связанных с дипломами, будет использоваться сущность “документ”, а для файлов самих работ “файл”.

- Сущность – **Диплом**. Атрибуты: Код\_диплома (является первичным ключом), Код\_студента (является внешним ключом), Код\_руководителя

(является внешним ключом), Тема\_диплома, Описание\_диплома, Статус, Дата\_создания, Дата\_обновления.

•Сущность – **Документ**. Атрибуты: Код\_документа (первичный ключ), Код\_диплома (внешний ключ), Код\_студента (внешний ключ), Путь\_к\_документу, Имя\_документа, Тип\_документа, Версия\_документа, Дата\_загрузки.

•Сущность – **Файл**. Атрибуты: Код\_файла (первичный ключ), Код\_диплома (внешний ключ), Код\_студента (внешний ключ), Путь\_к\_файлу, Имя\_файла, Тип\_файла, Версия\_файла, Дата\_загрузки.

Планируется, что будет возможность вести обсуждения хода работы, поэтому для этого нужны будут сущности, хранящие сообщения и другую информацию. Для этого будут использоваться сущности “обсуждение”, хранящее информацию об том, кто и что обсуждает, и “сообщение”, которое будет хранить, собственно сообщения для обсуждений.

•Сущность – **Обсуждение**. Атрибуты: Код\_обсуждения (первичный ключ), Код\_диплома (внешний ключ), Код\_создателя (внешний ключ), Тема, Дата\_создания.

•Сущность – **Сообщение**. Атрибуты: Код\_сообщения (первичный ключ), Код\_обсуждения (внешний ключ), Код\_отправителя (внешний ключ), Содержимое, Время\_создания.

Для обеспечения работы над проверками предварительных результатов практики, будут использоваться сущности “ревью” и “проверка норма контроля”.

•Сущность – **Ревью**. Атрибуты: Код\_ревью (первичный ключ), Код\_диплома (внешний ключ), Код\_проверяющего (внешний ключ), Комментарий, Оценка, Статус, Дата\_проверки.

•Сущность – **Проверка\_нормаКонтроля**. Атрибуты: Код\_проверки(первичный ключ), Код\_диплома (внешний ключ), Код\_проверяющего (внешний ключ), Комментарий, Статус, Отзыв, Дата\_проверки.

Для хранения итогов защиты будет использоваться сущность “итоговая оценка”.

•Сущность – **Итоговая\_оценка**. Атрибуты: Код\_диплома (первичный и внешний ключ), Код\_выставляющего (внешний ключ), Оценка, Оригинальность, Дата\_защиты.

#### 2.1.2 Набор связей сущностей

Для построения связи сущностей надо учесть следующие особенности:

- У пользователя может быть сколько угодно ролей, у роли может быть сколько угодно пользователей (связь многие-ко-многим);
- У студента может быть только одна группа, у группы может быть сколько угодно студентов (связь один-ко-многим);
- Диплом может курировать только один руководитель, руководитель может курировать сколько угодно дипломов (связь один-ко-многим);
- У диплома может быть только один студент, у студента может быть множество дипломов (связь один-ко-многим);
- Документ может принадлежать только к одному диплому, диплому может принадлежать множество документов (связь один-ко-многим);
- Документ может принадлежать только одному студенту, студенту может принадлежать множество документов (связь один-ко-многим);
- Файл может принадлежать только к одному диплому, диплому может принадлежать множество файлов (связь один-ко-многим);
- Файл может принадлежать только одному студенту, студенту может принадлежать множество файлов (связь один-ко-многим);
- Над дипломом может вестись сколько угодно обсуждений, одно обсуждение может вестись только над одним дипломом (связь один-ко-многим);
- У обсуждения может быть только один создатель, у создателя может быть множество обсуждений (связь один-ко-многим);

- В обсуждения может быть множество сообщений, одно сообщение может быть только в одном обсуждении (связь один-ко-многим);
- У сообщения может быть только один отправитель, у отправителя может быть множество сообщений (связь один-ко-многим);
- Одна итоговая оценка может быть только у одного диплома, у диплома может быть также только одна итоговая оценка (связь один-ко-одному);
- Итоговую оценку для одного диплома может выставить только один проверяющий, проверяющий может выставить итоговые оценки для множества дипломов (связь один-ко-многим);
- У диплома может быть множество проверок от нормаконтроля, нормаконтроль может проверять множество дипломов (связь многие-ко-многим).

### 2.1.3 Диаграммы «сущность-связь»

На основе имеющихся сущностей, их атрибутов, а также связей получают соответствующие диаграммы (рис. 1 - 4).

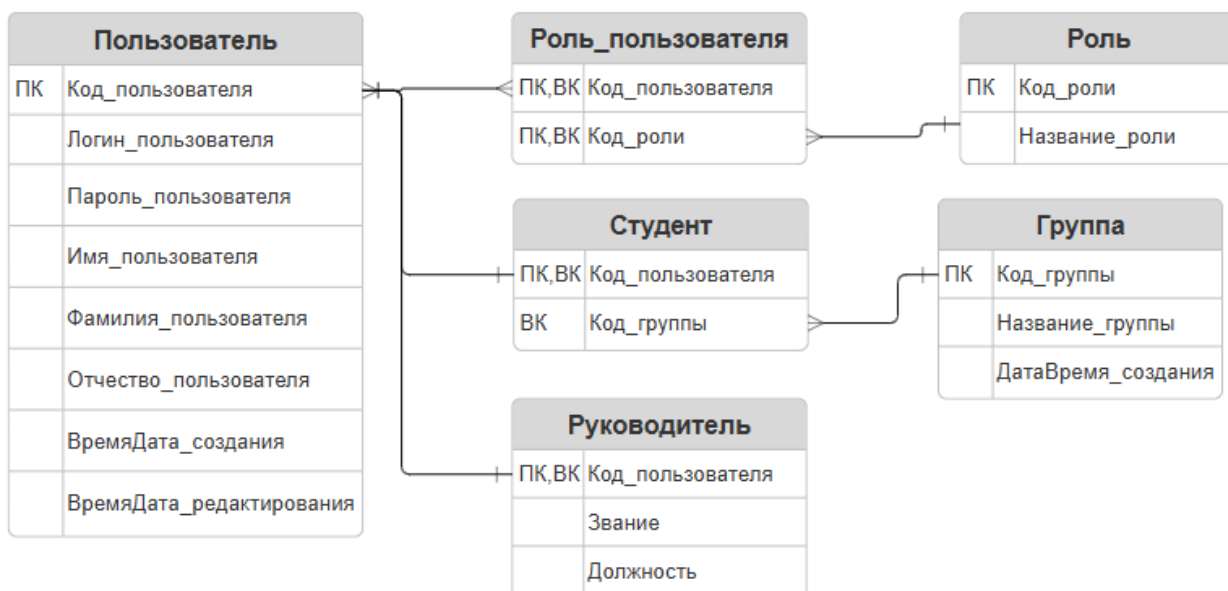


Рисунок 1 – Диаграмма «сущность-связь», относящаяся к пользователю

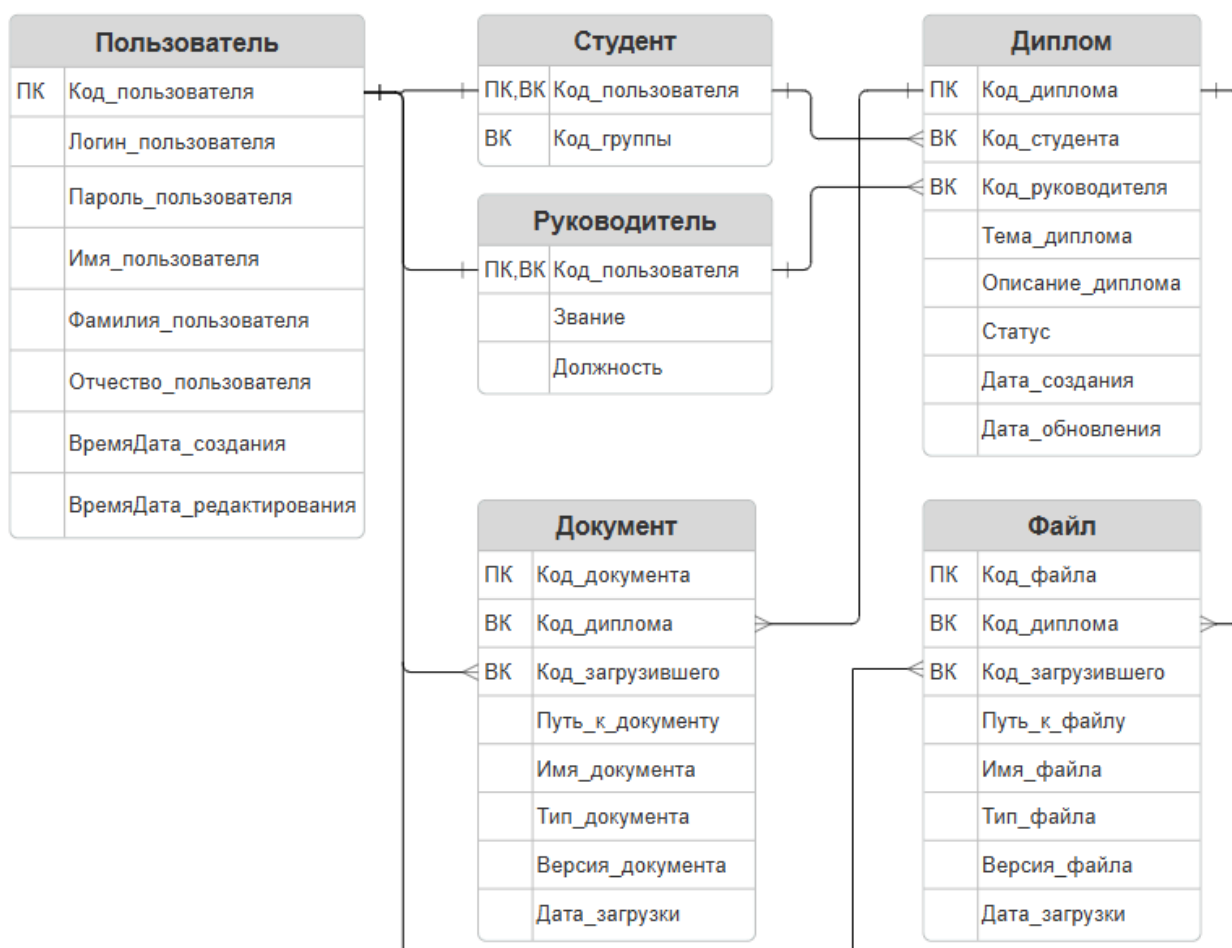


Рисунок 2 – Диаграмма «сущность-связь», относящаяся к дипломам

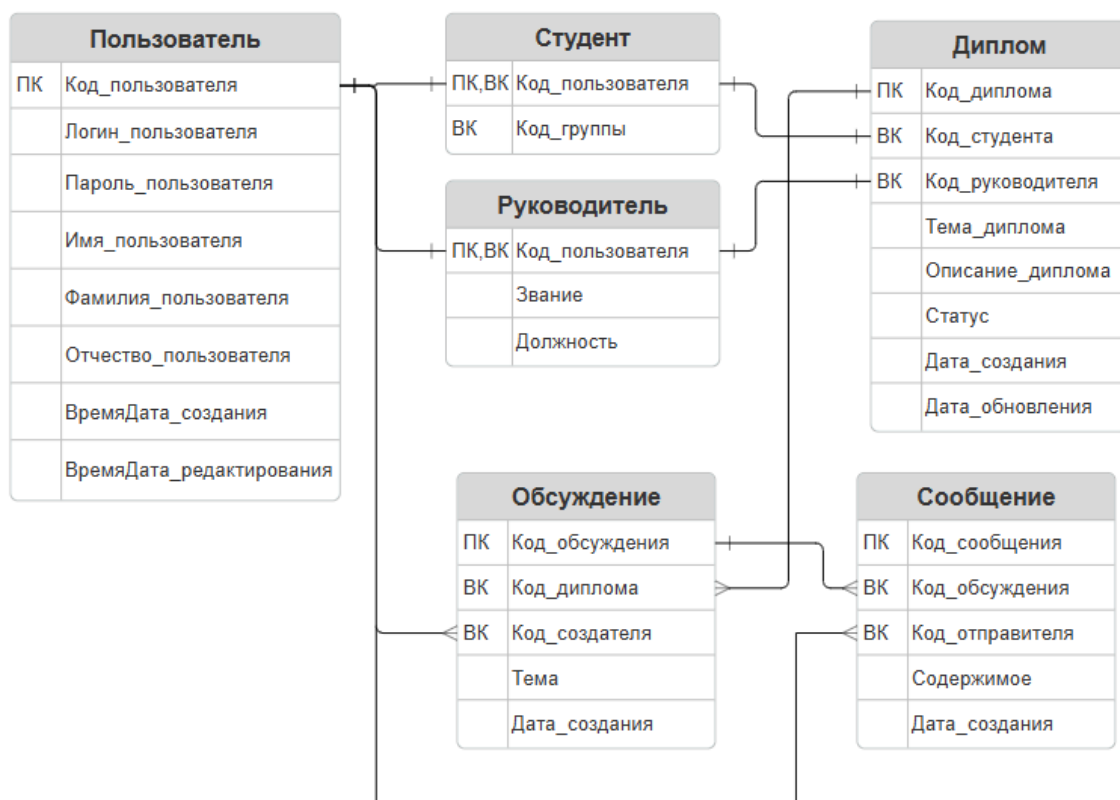


Рисунок 3 – Диаграмма «сущность-связь», относящаяся к обсуждениям





Рисунок 4 – Диаграмма «сущность-связь», относящаяся к проверкам и оценкам

#### 2.1.4 Наборы отношений

Из данной модели «сущность-связь» и её диаграмм получаем следующий набор отношений:

- Для сильной сущности *Пользователь* – отношение **Пользователи** (Код\_пользователя, Логин\_пользователя, Пароль\_пользователя, Имя\_пользователя, Фамилия\_пользователя, Отчество\_пользователя, ВремяДата\_создания, ВремяДата\_редактирования).
- Для сильной сущности *Роль* – отношение **Роли** (Код\_Роли, Название\_роли).

- Для слабой сущности *Роль\_пользователя* – отношение **Роли\_пользователей** (Код\_пользователя, Код\_роли).

- Для сильной сущности *Группа* – отношение **Группы** (Код\_группы, Название\_группы, ДатаВремя\_создания).

- Для слабой сущности *Студент* – отношение **Студенты** (Код\_пользователя, Код\_группы).

- Для слабой сущности *Руководитель* – отношение **Руководители** (Код\_пользователя, Звание, Должность).

- Для слабой сущности *Диплом* – отношение **Дипломы** (Код\_диплома, Код\_студента, Код\_руководителя, Тема\_диплома, Описание\_диплома, Статус, Дата\_создания, Дата\_обновления).

- Для слабой сущности *Документ* – отношение **Документы** (Код\_документа, Код\_диплома, Код\_студента, Путь\_к\_документу, Имя\_документа, Тип\_документа, Версия\_документа, Дата\_загрузки).

- Для слабой сущности *Файл* – отношение **Файлы** (Код\_файла, Код\_диплома, Код\_студента, Путь\_к\_файлу, Имя\_файла, Тип\_файла, Версия\_файла, Дата\_загрузки).

- Для слабой сущности *Обсуждение* – отношение **Обсуждения** (Код\_обсуждения, Код\_диплома, Код\_создателя, Тема, Дата\_создания).

- Для слабой сущности *Сообщение* – отношение **Сообщения** (Код\_сообщения, Код\_обсуждения, Код\_отправителя, Содержимое, Время\_создания).

- Для слабой сущности *Ревью* – отношение **Ревью** (Код\_ревью, Код\_диплома, Код\_проверяющего, Комментарий, Оценка, Статус, Дата\_проверки).

- Для слабой сущности *Проверка\_нормаКонтроля* – отношение **Проверки\_нормаконтроля** (Код\_проверки, Код\_диплома, Код\_проверяющего, Статус, Отзыв, Дата).

• Для слабой сущности *Итоговая\_оценка* – отношение **Итоговые\_оценки** (Код\_диплома, Код\_выставляющего, Оценка, Дата\_защиты).

#### 2.1.5 Нормализация отношений

Нормализация – это процесс преобразования отношений в БД к виду, отвечающему нормальным формам, с целью обеспечения минимальной избыточности (наличия лишних данных в таблицах) в этой базе данных.

Нормальные формы – это свойство отношений в БД, характеризующее с точки зрения избыточности, которое может привести к ошибочным результатам операций, проводящихся в БД.

Эдгардом Коддом и Рэймондом Бойсом были предложены шесть нормальных форм в пяти порядках [2,10]:

Условия для таблицы **первой нормальной формы** (1НФ) [2,3,10]:

- на пересечении каждой строки и каждого столбца содержится только одно значение;
- ни одно из ключевых полей не пусто.

Условия для таблицы **второй нормальной формы** (2НФ) [2,10]:

- удовлетворяют условиям 1НФ;
- нет неключевых полей, зависящих от части первичного ключа.

Условия таблицы **третьей нормальной формы** (3НФ) [2,10]:

- удовлетворяют условиям 2НФ;
- нет транзитивных зависимостей (нет атрибутов посредников между двумя атрибутами отношений).

Условия для таблицы **нормальной формы Бойса-Корда** (НФБК), которая является более строгим определением [2,10]:

- каждый детерминант (левая часть в функциональной зависимости атрибута А от атрибута В:  $A \rightarrow B$ ) отношения является потенциальным ключом.

Условия для таблицы **четвёртой нормальной формы** (4НФ) [2,10]:

- удовлетворяют условиям 3НФ;

- удовлетворяют условиям НФБК;
- нет многозначных зависимостей (многие-ко-многим).

Условия для таблицы **пятой нормальной формы (5НФ)** [2,10]:

- в таблице отсутствуют зависимые сочетания.

Во всех таблицах никакое из ключевых полей не может быть пустым, а также на каждом пересечении может содержаться только одно значение. Несмотря на то, что некоторые таблицы имеют поля для текста, значения в них атомарны, т.к. по отдельности они не будут нести смысла. Поэтому все таблицы удовлетворяют 1НФ.

У всех таблиц нет неключевых полей, зависящих от части первичного ключа, поэтому все таблицы удовлетворяют 2НФ.

Все таблицы, кроме таблиц «Документы» и «Файлы», не имеют транзитивных зависимостей. В этих двух таблицах, поля «Код\_студента» зависит от поля «Код\_диплома», т.к. дипломы привязаны к студентам и уже в них самих имеются коды студентов. Для приведения к 3-ей нормальной форме будет убран код студента.

Все таблицы удовлетворяют НФБК, т.к. у таблиц нет нескольких составных потенциальных ключей, имеющих общий атрибут.

4-ой нормальной форме удовлетворяют таблицы «Руководители», «Пользователи», «Группы», «Роли».

Дальнейшая нормализация не будет проводиться, т.к. их суть в улучшении целостности данных и уменьшения риска аномалий, что при текущем уровне считается минимальным.

Итого получается 14 отношений:

Пользователи: Код\_пользователя (первичный ключ),  
 Логин\_пользователя, Пароль\_пользователя, Имя\_пользователя,  
 Фамилия\_пользователя, Отчество\_пользователя, ВремяДата\_создания,  
 ВремяДата\_редактирования

Роли: Код\_роли (первичный ключ), Название\_роли.

Роли\_пользователей: Код\_пользователя (первичный и внешний ключ), Код\_роли (первичный и внешний ключ).

Группы: Код\_группы (первичный ключ), Название\_группы, ДатаВремя\_создания.

Студенты: Код\_пользователя (первичный и внешний ключ), Код\_группы (первичный и внешний ключ).

Руководители: Код\_пользователя (первичный и внешний ключ), Звание, Должность.

Дипломы: Код\_диплома (первичный ключ), Код\_студента (внешний ключ), Код\_руководителя (внешний ключ), Тема\_диплома, Описание\_диплома, Статус, Дата\_создания, Дата\_обновления.

Документы: Код\_документа (первичный ключ), Код\_диплома (внешний ключ), Путь\_к\_документу, Имя\_документа, Тип\_документа, Версия\_документа, Дата\_загрузки.

Файлы: Код\_файла (первичный ключ), Код\_диплома (внешний ключ), Путь\_к\_файлу, Имя\_файла, Тип\_файла, Версия\_файла, Дата\_загрузки.

Обсуждения: Код\_обсуждения (первичный ключ), Код\_диплома (внешний ключ), Код\_создателя (внешний ключ), Тема, Дата\_создания.

Сообщения: Код\_сообщения (первичный ключ), Код\_обсуждения (внешний ключ), Код\_отправителя (внешний ключ), Содержимое, Время\_создания.

Ревью: Код\_ревью (первичный ключ), Код\_диплома (внешний ключ), Код\_проверяющего (внешний ключ), Комментарий, Оценка, Статус, Дата\_проверки.

Проверки\_нормаконтроля: Код\_проверки (первичный ключ), Код\_диплома (внешний ключ), Код\_проверяющего (внешний ключ), Статус, Отзыв, Дата.

Итоговые\_оценки: Код\_диплома (первичный и внешний ключ), Код\_выставляющего (внешний ключ), Оценка, Дата\_защиты.

Когда будут создаваться таблицы для базы данных на основе имеющихся отношений, необходимо будет выбирать для них тип, размер, указывать могут ли они быть пустыми, значения по умолчанию, а также индексы для этих полей, чтобы на основе них создать ключи. Для кодов сущностей будут использоваться целочисленные неотрицательные переменные с автоинкрементом (int). Логины, пароли, имена и прочие поля, которые могут иметь различные символы, а не только буквы или цифры, будут иметь тип массива символов переменной длины (varchar). Для хранения времени и дат будет использоваться формат timestamp имеющий диапазон значений: от 1970-01-01 00:00:01 UTC до 2038-01-19 03:14:07 UTC. Пример формата таблицы «Пользователи» для MySQL, когда она будет составляться, приведен в таблице 1.

Таблица 1. Формат таблицы «Пользователи»

| Поле      | Тип         | NUL<br>L | Значение по<br>умолчанию | Дополнительно                 | Индекс  |
|-----------|-------------|----------|--------------------------|-------------------------------|---------|
| Код       | int(6)      | Нет      | Нет                      | AUTO_INCREMENT                | PRIMARY |
| Логин     | varchar(40) | Нет      | Нет                      |                               |         |
| Пароль    | varchar(32) | Нет      | Нет                      |                               |         |
| Имя       | varchar(30) | Нет      | Нет                      |                               |         |
| Фамилия   | varchar(30) | Нет      | Нет                      |                               |         |
| Отчество  | varchar(30) | Да       | NULL                     |                               |         |
| Создано   | datetime    | Нет      | current_timestamp()      |                               |         |
| Обновлено | datetime    | Нет      | current_timestamp()      | ON UPDATE CURRENT_TIMESTAMP() |         |

## 2.2 Проектирование веб-приложения

### 2.2.1 Стек разработки

В качестве языка программирования для написания веб-приложения выбран JavaScript - мультипарадигменный язык программирования, который поддерживает объектно-ориентированный, императивный и функциональный стили. Он является хорошим выбором как для front-end, так и back-end разработки веб-приложений благодаря своей универсальности, экосистеме и производительности. Использование платформы Node.js позволяет разрабатывать и клиентскую, и серверную части на одном языке, что сокращает время обучения и упрощает поддержку кода. Платформа работает на движке V8 (как Chrome) [11], обеспечивая быструю обработку запросов благодаря асинхронной, событийно-ориентированной архитектуре. Имеется крупный репозиторий пакетов с готовыми решениями- npm – что ускорит разработку и снизит затраты на написание кода с нуля. Так же JavaScript нативно работает с JSON — стандартом обмена данными в вебе. Это упрощает интеграцию с front-end (React, Angular) и сторонними API.

В качестве среды разработки используется Visual Studio Code от компании Microsoft – легкий редактор кода для кроссплатформенной разработки веб -и облачных приложений. Имеет в себе отладчик, инструменты для работы с Git, который используется для контроля версий, подсветку синтаксиса, технологию авто-дополнения IntelliSense и средства рефакторинга.

### 2.2.2 Пользовательский интерфейс

Для проектирования пользовательского интерфейса использовался онлайн-сервис для разработки дизайнов и прототипирования Figma, с бесплатным ограниченным функционалом. Пример главной страницы заведующего-кафедрой, сделанный в Figma можно посмотреть на рисунке 5.

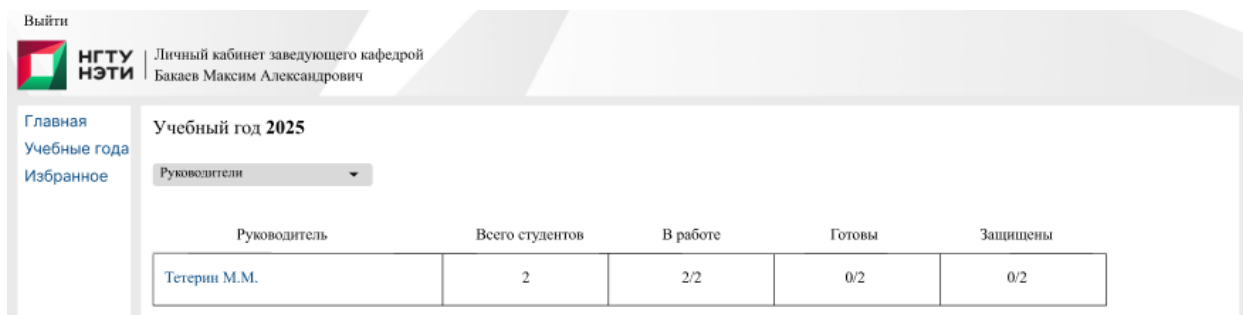


Рисунок 5 –Макет интерфейса личного кабинета заведующего кафедрой.

Для описания разметки самих страниц будет использоваться EJS (Embedded JavaScript) [12]. Это шаблонная система на основе JavaScript, которая позволяет верстать разметку страниц со встроенным кодом JavaScript, а также там имеется возможность собирать эти страницы HTML на основе нескольких шаблонов сразу, вместо их целого хранения. Например, можно хранить разметку шапки, центральную части и подвала отдельно, а потом собрать их в один html-документ и отправить пользователю, что упрощает редактирование отдельных элементов без необходимости правки каждой страницы в отдельности.

### 2.2.3 Взаимодействие с базой данных

Для взаимодействия с базой данных приложение будет обращаться к СУБД MySQL. Для этого используется библиотека mysql2[13]. При запуске сервера, она подключается к БД на основе настроек конфигурационного файла. После подключения, приложение может посылать SQL-запросы в СУБД, например чтения или вставки новой записи таблицы.

### 2.2.4 Скачивание и загрузка файлов

Для этой цели используется пакет multer[14], который позволяет скачивать и загружать файлы из форм. В текущих рамках приложения можно загружать документы только форматов .pdf и .docx, а также архивы .zip и .rar. К загрузке файлов в дипломные проекты допущены только пользователи с ролями студентов и научных руководителей. Загруженные файлы хранятся с изначальными форматами в специальной директории на сервере, с



измененными именами, а информация об файлах помещается в специальные таблиц БД.

### 2.2.5 Роли пользователей

Для разграничения прав и возможностей пользователей в системе предусмотрена ролевая модель. Всего в системе 5 ролей: студент, научный руководитель, нормаконтроль, заведующий кафедрой и администратор. Информация о том, какая роль у пользователя хранится в таблице `user_roles` (см. соответствующий раздел проектирования БД), которая запрашивается системой при авторизации пользователя. Каждая из ролей имеет свой личный кабинет с набором функций:

- Студент видит свои работы, обсуждения этих работ и проверки. Он может добавлять новые работы, загружать и скачивать для этих работ файлы и документы через специальные формы, и удалять их, писать сообщения в обсуждениях с научным руководителем и нормаконтролем, а ещё отправлять ему или нормоконтролю работы на проверки. На странице работ, студент может видеть список задач, которым управляет его научный руководитель. Пример макета личного кабинета студента представлен на рисунке 6.

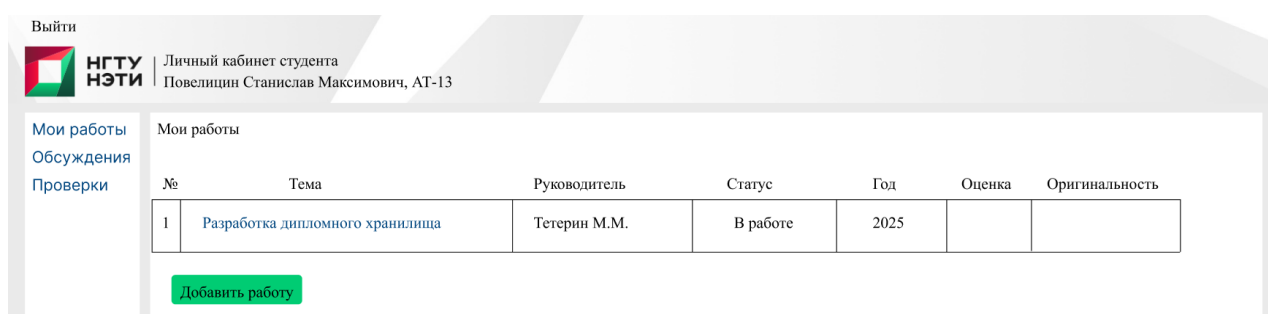


Рисунок 6 – Макет личного кабинета студента.

- Научный руководитель видит работы студентов, которые он когда-либо курировал, обсуждения этих работ и их проверки. На странице работы он может скачивать и загружать, как и студент, документы и файлы

для них, может менять статус работе (в работе, одобрено, защищено, на проверке, требуется доработка) и настраивать список задач для студента. На странице проверки имеется возможность скачать проверяемый файл, написать отзыв, указать статус проверки (проверено, отклонено) и прикрепить, если необходимо, ответный файл. В разделе обсуждений видеть все обсуждения со студентами, а в конкретном обсуждении оставлять сообщения. Пример макета обсуждений приведен на рисунке 7.

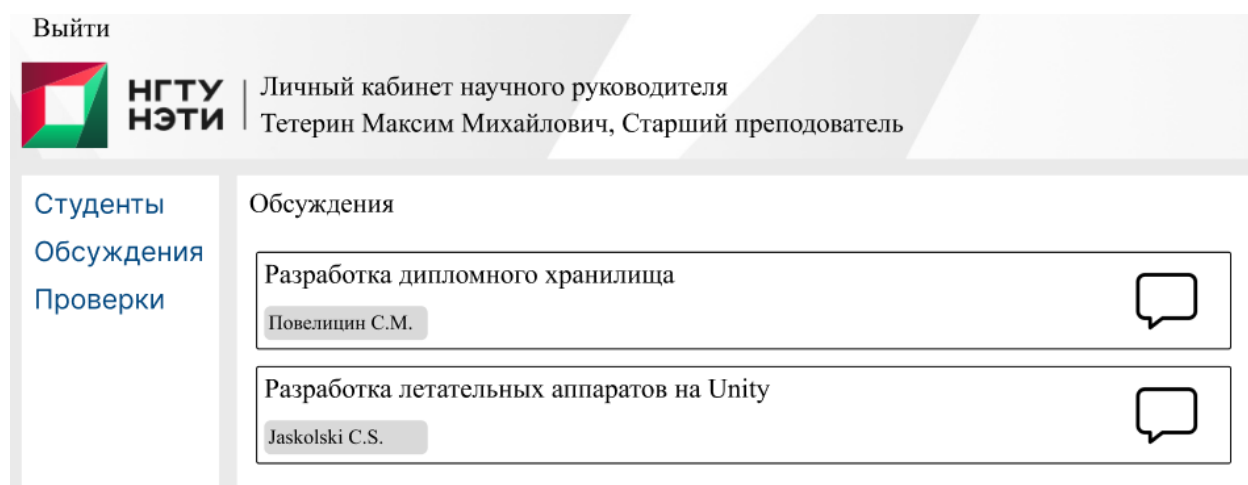


Рисунок 7 – Макет страницы обсуждений.

- Нормаконтроль имеет в своем кабинете страницы обсуждений и проверок. На странице проверки имеется возможность скачать проверяемый документ, написать отзыв, указать прошёл или нет документ проверку и прикрепить, если необходимо, ответный документ. Функционал обсуждений идентичен научному руководителю. Макет страницы проверок приведен на рисунке 8.

| Студент        | Тема                            | Статус    | Оценка | Комментарий                                                         |
|----------------|---------------------------------|-----------|--------|---------------------------------------------------------------------|
| Повелицин С.М. | Разработка дипломного хранилища | Отклонено | 0      | ТЕКСт Текст фывфывфы<br>оаоаоаоыфпрвлфвоыл                          |
| Повелицин С.М. | Разработка дипломного хранилища | Отклонено | 100    | aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa<br>aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa... |
| Повелицин С.М. | Разработка дипломного хранилища | Ожидает   |        |                                                                     |

- Заведующий кафедрой имеет в своем кабинете статистику того, сколько студентов есть руководителей или групп, сколько их работ находят в процессе написания, сколько готовы и сколько из них защищено за текущий учебный год, пример макета приведен на рисунке 9. Может просмотреть эту статистику за другие года, а также просмотреть избранные работы. Заведующий кафедрой может просмотреть списки студентов у научных руководителей или групп, а ещё сами работы, а также добавить их или удалить из избранного.

### Руководители

| Руководитель | Всего студентов | В работе | Готовы | Защищены |
|--------------|-----------------|----------|--------|----------|
| Тетерин М.М. | 2               | 2/2      | 0/2    | 0/2      |

- Администратор имеет возможность просмотреть статистику похожую на ту, что есть у заведующего кафедрой; может смотреть списки студентов у научных руководителей и групп; создавать и удалять

пользователей и группы, и наделять пользователей ролями; создавать работы и удалять, но не может загружать файлы.

#### 2.2.6 Асинхронность

Асинхронность играет важную роль в системе. Действия пользователей не должны приводить к блокировкам ресурсов системы. Для этого в Node.js предусмотрены асинхронные функции `async` и `await`. С их помощью написан асинхронный код виде, т.к. после вызова функции система не ждет их завершения для выполнения следующих команд.

#### 2.2.7 Авторизация

При первом заходе пользователя будет встречать форма авторизации, в которую нужно будет ввести логин и пароль. Далее приложение запросит данные по логину в БД. Если логин неверен, то будет выдано сообщение о том, что такого пользователя нет. При неправильном пароле будет выведено сообщение о том, что он неверен, а также начнется подсчет неправильных попыток, по истечении которых, возможность авторизации по данному логину будет временно приостановлено. Если все введенные данные будут корректны, то пользователю будет выдан токен доступа с определенным сроком жизни, а потом пользователь будет перенаправлен в личный кабинет. Токен доступа создан на основе JWT (JSON Web Token) [15] – открытом стандарте для создания токенов доступа, основанном на формате JSON. Для их создания используется библиотека `jsonwebtoken` [16].

#### 2.2.8 Почтовая служба

Для того, чтобы пользователям постоянно не приходилось проверять личные кабинеты на предмет новых сообщений или проверок, должна быть система уведомлений на почту. Для этого будет использоваться библиотека `Nodemailer` [17]. Для нее необходим почтовый сервис, с которого будут формироваться письма, и логин с паролем от этого сервиса. После отправки сообщения в обсуждение должно формироваться письмо, которое сообщает о том, что пришло новое сообщение с текстом сообщения, отправителем и обсуждением. При отправке студентами работ на проверку научным

руководителям и нормаконтролю должно высылаться оповещение на почту о том, что студент запрашивает проверку по своей работе с указанием имени студента и темы работы. При завершении проверки научными руководителями и нормаконтролем студенту должно прийти уведомление, что проверка завершена и указать вид проверки, проверяющего и результат с отзывом.

#### 2.2.9 Обсуждения

При создании дипломной работы, между студентом и научным руководителем, и студентом и нормаконтрлем будут автоматически создаваться обсуждения, в котором они смогут общаться. Просмотр и отправка сообщений должны быть в отдельном окне. Сообщения должны быть ограничены только текстом и максимумом в 1000 символов.

Уведомления об новых сообщениях должны высылаться на почту.

#### 2.2.10 Проверки

Студент может отправить свою работу на проверку научному руководителю или нормаконтролю используя отдельную форму. В ней он может выбрать тип проверки, работу и написать комментарий. После подтвердить отправку формы.

При проверке работ у научного руководителя и нормаконтроля должны отображаться ФИО студента, тема работы, его комментарий, а также файл, отправленный на проверку. Все файлы можно скачать. После проверки можно указать её статус: принято, отклоннено, написать ответный комментарий для пояснения и прикрепить файл. Нормаконтроль на проверку может получать только текстовые документы. Может создать у себя в личном кабинете чек-лист, который будет отображаться при проверке документов. При проверке он может сделать запрос к API генеративной модели для проверки документа на соответствие ГОСТам.

#### 2.2.11 Навигация

Для навигации по сайту будет использоваться боковое меню, на котором будут отображаться основные вкладки роли. Дополнительные

ссылки будут использоваться для перехода на страницы с информацией об пользователе (списки студентов у научного руководителя или ученой группы) или об студенческой работе (для просмотра её состояния или скачиванию-загрузке файлов).

На основе разработанных базы данных и функционала веб-приложения было создано непосредственно само приложения. Далее будет подробнее рассмотрены ключевые алгоритмы, реализованные в коде приложения с пояснениями.

### 3. РЕАЛИЗАЦИЯ

В данной главе описывается реализация основных компонентов разработанной системы. Будут подробно описаны алгоритмы ключевых функций с фрагментами программного кода, показывающими решение поставленных задач. В данной главе не будут отображены в полной мере созданные на основе раздела 2.1 таблицы базы данных.

#### 3.1. База данных

При разработке приложения оказалось, что таблиц, созданных на основе раздела 2.1 оказалось недостаточно или они не удовлетворили полностью реализации функций. Для этого было создано ещё несколько таблиц:

- `discussion_user` – таблица для связи между обсуждениями и пользователями, т.к. старый формат плохо подходил для связи двух пользователей, но с новой таблицей стало возможно создание обсуждения на несколько пользователей;
- `favorites` – таблица для хранения измарианных работ, т.к. заведующий кафедрой хотел бы иметь такую возможность. Связывает пользователей и объекты дипломов;
- `review_documents` – таблица для хранения информации об документах посылаемых на ревью. Старая таблица ревью плохо поддерживала возможность прикрепления нескольких документов к ревью;
- `review_files` – таблица для хранения информации об файлах, присылаемых в ревью. Со старой таблицей не было возможности прикреплять никакие другие файлы, кроме документов.

### 3.2. Вспомогательные функции

Ниже перечислены некоторые вспомогательные функции, используемые неоднократно в процессе работы программы:

- `authenticateToken` – функция, отвечающая за проверку токенов пользователей. Она получает токен из куки браузера, если он отсутствует, то переводит пользователя на страницу авторизации, с сохранением изначального адреса. Вызывает функцию `verify` из библиотеки `jwt`, которая сравнивает значения секретных ключей токена и системы, если возникает ошибка, то токен очищается из куки и пользователя перевод на страницу авторизации. Затем передает управление дальше в систему.
- `checkRole` – функция, отвечающая за проверку роли пользователя. На вход получает массив ролей. Если массив пустой, то выведется ошибка 500, что нет ролей. Если нужной роли не окажется, то выведется ошибка 403 “Доступ запрещен: нет уровня допуска”.

### 3.3. Ключевые алгоритмы

#### 3.3.1. Авторизация пользователей

Этот алгоритм отвечает за авторизацию пользователей в системе на основании логина и пароля. Последовательность действий следующая:

- 1) Пользователь вводит логин и пароль в соответствующие поля формы, и нажимая на кнопку “Войти” отправляет запрос на сервер;
- 2) Система извлекает значения логина и пароля из тела запроса в переменные `login` и `password`;
- 3) Система ищет пользователя по указанному логину. Для этого формируется запрос в СУБД на выборку из таблицы `users` всех записей со всеми полями, содержащих указанный логин, а затем берется первая запись и



записывается в переменную user. Если такой записи нет, то выводится сообщение о том, что логин или пароль неверен;

4) Система проверяет правильность пароля для пользователя. Для этого сравниваются значения переменной password и значения поля password\_hash. Если значения не совпадают, то выводится ошибка о том, что логин или пароль неверен, и увеличивается счетчик неправильных попыток;

5) После того, как все данные будут верны, делается запрос в СУБД на поиск роли пользователя в таблице user\_roles по значению кода пользователя user.id (см. рис 10);

6) Далее создается JWT-токен со сроком жизни 8 часов, содержащий логин, пароль, роль, а также секретный ключ, который затем заносится в куки. Этот токен используется для последующих автоматических авторизаций с браузера, с которого зашел пользователь;

7) Затем пользователь перенаправляется в свой личный кабинет в зависимости от его роли.

```
// 1. Поиск пользователя
const [users] = await pool.query('SELECT * FROM users WHERE email = ?', [login]);
if (users.length === 0) {
  return res.status(401).render('login', { error: 'Неверный логин или пароль' });
}
const user = users[0];
// 2. Проверка пароля
if (password !== user.password_hash) {
  return res.status(401).render('login', { error: 'Неверный логин или пароль' });
}
// 3. Получаем роли пользователя
const [roles] = await pool.query(`
  SELECT r.name
  FROM user_roles ur
  JOIN roles r ON ur.role_id = r.id
  WHERE ur.user_id = ?
`, [user.id]);
```

Рисунок 10 – Фрагмент кода авторизации из пунктов 2 – 5.

### 3.3.2. Создание репозитория ВКР

Данный алгоритм отвечает за создание объекта, связанного с хранением информации и файлов ВКР. Возможность создавать репозитории имеют студенты и научные руководители. Шаги следующие:

1) Пользователь, студент или научный руководитель, нажимает на кнопку добавления работы, в результате вызывается обработчик `app.get(/new-diploma)`;

2) Обработчик сначала проверяет токен пользователя, вызывая функцию `authenticateToken`, которая забирает токен из куки браузера и проверяет его, а затем вызывается метод `checkRole`, определяющий роль пользователя. Если токен и роль пользователя проходят проверку, то обработка переходит к основной последовательности;

3) После вызова обработчик получает данные обо всех научных руководителях методом `getSupervisors` и студентах методом `getStudents`, а также информацию об вызывающем пользователе через `req.user`, после чего, в зависимости от того, является вызывающий студентом или научным руководителем, пользователя переводит на страницу `student/new-diploma` или `supervisor/new-diploma`;

4) В вызванной форме пользователь должен выбрать из выпадающего списка научного руководителя (если пользователь - студент) или студента (если пользователь – научный руководитель) и заполнить поле с названием темы ВКР. Также можно написать краткое описание работы;

5) После нажатия на кнопку создания работы, вызывается обработчик отправки формы `app.post(/student/new-diploma)` или `app.post(/supervisor/new-diploma)` в зависимости от того, что за пользователь его вызвал. Также происходит вызов методов `authenticateToken` и `checkRole`, перед остальной обработкой;

6) В начале происходит валидация введенных данных, проверяющих заполненность полей темы, студента и научного руководителя. Если данные

незаполнены, то появляется ошибка, сообщающая о том, что необходимо заполнить эти поля;

7) Далее на основе значений полей формы title, description, supervisor\_id и student\_id, а также значения id пользователя из запроса req.user.id, происходит insert-запрос в СУБД, для создания нового объекта ВКР, который возвращает значение поля id новой записи, записываемое в переменную diploma\_id;

8) Потом вызываются два обработчика создания обсуждений Первый вызов нужен для создания обсуждения между научным руководителем и студентом, второй между нормаконтролем и студентом. В оба вызова подаются значения diploma\_id и title, а также в первый подаются supervisor\_id и student\_id, а во второй только student\_id и результат запроса в СУБД на поиск нормаконтролёра normcontrol\_id (см. рис 11);

9) Если при создании нового ВКР произошла ошибка, то вызывается обработчик, который сообщает пользователю о том, что работу не удалось создать, а также записывающий её в файл лога.

```
// Создание новой работы
const [result] = await pool.query(`
  INSERT INTO diplomas (student_id, supervisor_id, title, description, status)
  VALUES (?, ?, ?, ?, 'draft')
`, [req.user.id, supervisor_id, title, description || null]);

const diplomaId = result.insertId; //Получение ИД только что созданной работы
// Создание обсуждения с руководителем
const supDiscussionId = await createDiscussion(
  diplomaId,
  `Обсуждение работы "${title}"` научным руководителем`
);
// Добавляем участников (студент и научный руководитель)
await addUserToDiscussion(supDiscussionId, req.user.id);
await addUserToDiscussion(supDiscussionId, supervisor_id);
// Поиск нормаконтроля для вставки
const [normControl] = await pool.query(`
  SELECT user_id FROM user_roles WHERE role_id = (
    SELECT id FROM roles WHERE name = 'norm_control'
  ) LIMIT 1
`);
```

Рисунок 11 – фрагмент кода создания ВКР

### 3.3.3. Загрузка файлов в репозиторий ВКР

Данный функционал отвечает за загрузку пользователями документов и файлов, относящихся к ВКРу. В основном используется обработчик Алгоритм состоит из следующих шагов:

1) Пользователь нажимает на кнопку “Выбрать файл”, после чего открывается диалоговое окно, в котором пользователь должен выбрать файл для загрузки. Окно, настроенное на прием файлов форматов: .pdf, .doc, .docx, если это загрузка документов, и форматов: .rar, .zip, если загрузка файлов;

2) Далее он нажимает кнопку “Загрузить” и вызывается обработчик `app.post(/upload/document)`, если это документ, или `app.post(/upload/file)`, если это архив проекта. В теле запросов также имеются функции `authenticateToken` для проверки токена, `checkRole` с параметрами 'student','supervisor' для проверки того, что пользователь является студентом или научным руководителем, и метод `upload.single` с параметрами 'document' или 'file' в зависимости от того, что загружается, который загружает файлы в свои директории хранилища. Для того, чтобы не было перезаписи файлов к ним добавляется “соль” состоящая из случайного числа и даты загрузки по UTC. После выполнения переходит в тело функции;

3) В теле функции в переменную `originalName` записывается имя файла из буфера в кодировке `utf8`, которая будет использоваться для передачи имени файлов в запрос;

4) Затем формируется `insert`-запрос в СУБД на вставку новой записи в таблицу `documents` или `files`, для хранения информации об файле. Значения для полей берутся из тела запроса, кроме изначального названия файла, который берется из переменной `originalName` (см. рис 12);

```
// Получаем оригинальное имя файла с правильной кодировкой
const originalName = Buffer.from(req.file.originalname, 'latin1').toString('utf8');
await pool.query(
  'INSERT INTO documents (diploma_id, user_id, file_path, file_name, file_type) VALUES (?, ?, ?, ?, ?)',
  [req.body.diploma_id, req.user.id, req.file.path, originalName, req.file.mimetype]
);
```

Рисунок 12 – фрагмент кода загрузки документа

5) Затем отправляется уведомление на почту студенту или научному руководителю о том, что в дипломную работу был загружен новый файл;

6) После этого пользователя возвращает на обновленную страницу работы.

#### 3.3.4. Создание обсуждений

Данная функция обычно вызывается, когда пользователь создает новый объект диплома. Алгоритм следующий:

1) Вначале вызывается функция `createDiscussion`, чтобы создать сам объект обсуждения в базу данных. Функция считывает значение переменных `diploma_id` и `title`;

2) После этого формируется запрос в СУБД на вставку новой записи в таблицу `discussions`, в который передают переменные `diploma_id` и `title`. Результатом этой вставки является `id` созданного объекта, сохраняемого в переменную `discussion_id`. Если при создании появилась ошибка, то отобразится соответствующее сообщение (см рис. 13);

```
async function createDiscussion(diploma_id, title) {
  try {
    //Вставка в таблицу нового обсуждения
    const [result] = await pool.query(
      `INSERT INTO discussions (diploma_id, title)
      VALUES (?, ?)`,
      [diploma_id, title]
    );
    return result.insertId; // Возвращаем ID созданного обсуждения
  } catch (error) {
    console.error('Ошибка при создании обсуждения:', error);
    throw error;
  }
}
```

Рисунок 13 – Фрагмент кода функции создания обсуждений

3) Далее в обсуждение вставляются 2 пользователя. Для этого два раза вызывается функция `addUserToDiscussion`, которая на вход получает `discussion_id` и `user_id`, на основе которых должна будет сделаться новая

запись в таблицу `discussion_user`. Если появляется ошибка, то вылезет сообщение об этом.

### 3.3.5. Запрос на проверку работы

Этот алгоритм отвечает за то, когда студент хочет отправить свою работу на проверку научному руководителю для оценки хода работы или нормоконтролю для оценки оформления пояснительной записки. Шаги алгоритма следующие:

1) Студент нажимает на кнопку создания проверки и его переводит на страницу создания заявки на проверку, вызывая обработчик `app.get(/student/check)` (см. рис. 14);

```
// Страница создания заявки на проверку
app.get('/student/check', authenticateToken, checkRole('student'), async (req, res) => {
  try {
    // Получаем работы студента
    const [diplomas] = await pool.query(`
      SELECT id, title FROM diplomas
      WHERE student_id = ? AND status != 'defended'
    `, [req.user.id]);

    // Получаем возможных проверяющих (научные руководители и нормоконтроль)
    const [reviewers] = await pool.query(`
      SELECT u.id, u.first_name, u.last_name, r.name as role
      FROM users u
      JOIN user_roles ur ON u.id = ur.user_id
      JOIN roles r ON ur.role_id = r.id
      WHERE r.name IN ('supervisor', 'norm_control')
    `);
```

Рисунок 14 – фрагмент кода обработки события `app.get(/student/check)`

2) На странице он из раскрывающихся список заполняет поля “Тип” с идентификатор `type`, “Работа” с идентификатором `diploma_id`, “Проверяющий” с идентификатором `reviewer_id`, “Файлы”, являющимся массивом идентификаторов из таблиц `documents` и `files`, и, если надо, пишет комментарий в поле “Комментарий”, с идентификатором `student_comment`. После чего нажимает на кнопку “Отправить”. Если поля “Работа”,

“Проверяющий” и “Файлы” не заполнены, то появляется сообщение о том, что их надо заполнить;

3) Вызывается обработчик `app.post(/student/check)`, который вызывает функции `authenticateToken` и `checkRole` для проверки токена и роли студента. Далее происходит дальнейшая обработка;

4) Из тела запроса считываются значения всех полей в переменные: `type` для определения типа проверки, `diploma_id` для получения объекта ВКР, `reviewer_id` для указания проверяющего и `file_id` с `document_id` для поиска выбранных файлов, а также `student_comment` для сохранения сообщения студента;

5) В зависимости от того, что это будет за проверка, создается запрос в СУБД для вставки новой записи или в таблицу `reviews`, если проверка у руководителя, или в таблицу `norm_control_checks`, если у нормаконтроля, используя переменные для определения значений полей. Также делаются соответствующие запросы на вставку записей в таблицы `review_documents` и `review_files`, если проверка делается у научного руководителя. Если вставка не удалась, то появляется сообщение о том, что создание проверки не удалось

### 3.3.6. Проверка работы научным руководителем

Данный функционал отвечает за обработку присылаемых студентами проверок, проводимых научным руководителем для оценки хода работы студента.

1) Научный руководитель нажимает на кнопку “Проверить”, вызывая обработчик `app.get(/supervisor/review/:id)` для получения данных для отрисовки страницы проверки. Он вызывает сначала функции `authenticateToken` и `checkRole` для получения токена и проверки того, что пользователь имеет роль научного руководителя;

2) Потом в СУБД делается запрос на выборку из таблиц `users`, `students`, `diplomas`, `reviews`, `review_documents` и `review_files` всех записей, связанных с данной проверкой на основе значения поля `id` таблицы `reviews` равной значению `id` запроса;

- 3) После этого пользователя переводит на страницу `supervisor/review`, в которую передаются собранные данные;
- 4) Если на этом этапе обработки `get` запроса возникает ошибка, то выводится сообщение об этом и дальнейшая работа прекращается;
- 5) На странице самой проверки руководитель может скачать прикреплённые файлы, вызвав обработчики `app.get(/download/document/:id)` или `app.get(/download/files/:id)` соответственно;
- 6) Далее научный руководитель должен заполнить поля “Оценка” идентификатором `score`, “Отзыв” – `feedback`, выбрать значение поля “Результат” – `status` и прикрепить ответный документ – `document`. Обязательным является только поле “Результат”. Если оно не выбрано, то вылезет сообщение о том, что необходимо выбрать результат проверки;
- 7) При нажатии на кнопку “Завершить” вызывается обработчик `app.post(/supervisor/review)`. В начале также вызываются функции `authenticateToken` и `checkRole`, а также `uploadDocument.single('document')` для загрузки прикрепленного ответного документа;
- 8) Потом идет считывание из тела запроса значений полей `score`, `feedback` и `status` и запись их в переменные `score`, `feedback` и `status` соответственно, а также сохранения оригинального имени ;
- 9) Делается запрос в СУБД на обновление полей `score`, `status`, `feedback`, `response_document` записи таблицы `reviews`, значение поля `id` которой равен `id` из `post`-запроса;
- 10) Затем делается запрос на добавление новой записи в таблицу `review_documents` для сохранения информации об ответном документе;
- 11) После этого вызывается команда `res.redirect('/supervisor/reviews')` для перевода пользователя на страницу с проверками;
- 12) Если на этом этапе появилась ошибка, то выведется сообщение о том, что обработка не удалась.



### 3.3.7. Проверка работы нормаконтролем

Данный функционал отвечает за обработку присылаемых студентами проверок, проводимых нормаконтролем для оценки пояснительных записок студентов.

1) Нормаконтроль нажимает на кнопку “Проверить”, вызывая обработчик `app.get(/norm_control/norm_control_check/:id)` для получения данных для отрисовки страницы проверки. Он вызывает сначала функции `authenticateToken` и `checkRole` для получения токена и проверки того, что пользователь имеет роль нормаконтроля;

2) Потом в СУБД делается запрос на выборку из таблиц `users`, `students`, `diplomas` и `norm_control_checks` всех записей, связанных с данной проверкой на основе значения поля `id` таблицы `norm_control_checks` равной значению `id` запроса;

3) После этого пользователя переводит на страницу `norm_control/norm_control_check`, в которую передаются собранные данные;

4) Если на этом этапе обработки `get` запроса возникает ошибка, то выводится сообщение об этом и дальнейшая работа прекращается;

5) На странице проверки нормаконтроль может скачать прикрепленный документ, вызвав обработчики `app.get(/download/document/:id)`;

6) Далее нормаконтроль должен выбрать значение поля “Допуск” с идентификатором `is_passed`. Если необходимо заполнить поле “Отзыв” – `feedback`, и прикрепить ответный документ – `document`. Если значение “Допуск” не выбрано, то вылезет сообщение о том, что необходимо выбрать результат проверки;

7) При нажатии на кнопку “Завершить” вызывается обработчик `app.post(/norm_control/norm_control_check)`. В начале также вызываются функции `authenticateToken` и `checkRole`, а также `uploadDocument.single(`document`)` для загрузки прикрепленного ответного документа;

8) Потом идет считывание из тела запроса значений полей `feedback` и `is_passed` и запись их в переменные `feedback` и `is_passed` соответственно, а также сохранение оригинального имени ответного документа в переменную `originalName`;

9) Делается запрос в СУБД на основе переменных для обновления полей `feedback`, `is_passed` и `respond_file` записи таблицы `norm_control_checks`, значение поля `id` которой равен `id` из `post`-запроса;

10) Если на этом этапе появилась ошибка, то выведется сообщение о том, что обработка не удалась.

Приведенные выше алгоритмы обеспечивают основной функционал разработанной системы. На их основе выполняется авторизация пользователей, создание ими новых объектов ВКРов, загрузку и выгрузку данных, связанных с этими ВКРами, ведение обсуждений и проверок работ. Реализация выполнена на основе языка JavaScript с применением фреймворка Node.js и библиотек для него, для достижения поставленных задач.

## ЗАКЛЮЧЕНИЕ

Целью данной работы было создание системы для организованного хранения, сопровождения и демонстрации ВКР студентов. Результатом стала разработанное веб-приложение.

Перед началом работ были собраны пожелания будущих пользователей данного приложения, для определения задач, которые разрабатываемая система будет решать. На основе этого был проведен поиск существующих решений, но продуктов, полностью удовлетворяющих требованиям, не был найден, т.к. функционал имелся или только для работы и оценки документов (у «ВКР-ВУЗ»), или только для работы с программным обеспечением (у «GitLab»).

Для разработки решения был собран список литературы, на основе которого затем была спроектирована база данных, а затем создано решение в виде веб-приложения. При создании непосредственно приложения, база данных получилась сложнее, чем планировалась, т.к. проект, на основе которого она создавалась, не удовлетворял всему функционалу, который разрабатывался. Из чего можно сделать вывод, что сначала необходимо было проработать функционал приложения, а потом для него разрабатывать базу данных.

При проектировании базы данных были сначала созданы сущности и связи между ними, на основе которых в последствии должны были быть созданы таблицы для хранения данных. Для обеспечения стабильности хранения данных и использования базы данных была проведена нормализация до 3-го уровня по Рэймонду и Кодду. На основе пожеланий и задач были спроектированы модули приложения, которые вместе с базой данных должны были составлять разрабатываемое веб-приложение.

Как писалось выше, итоговая база данных содержит на 4 таблицы больше, чем проектировалось, при условии ещё не полной реализации всего запланированного функционала. Реализованный функционал состоит из

возможности создания студентами и научными руководителями объектов ВКРа, загрузки в них документов и архивов; в возможности вести диалоги между студентами, научными руководителями и нормоконтролем; студенты могут отправлять работы на проверку руководителям и нормоконтролю. Для заведующего кафедрой была реализована возможность добавлять работы и избранное, а также отображение статистики прогресса студентов у научного руководителя и группы.

## СПИСОК ЛИТЕРАТУРЫ

1. VKR-Smart [Электронный ресурс]. – URL: <https://dev.vkr-smart.ru> (дата обращения: 14.03.2025)
2. Базы данных. Проектирование, реализация и сопровождение. Теория и практика. 3-е издание.: Пер. с англ. — М. : Издательский дом "Вильямс", 2003. — 1440 с.
3. Введение в реляционные базы данных / В. В. Кириллов, Г. Ю. Громов. — СПб.: БХВ-Петербург, 2009. — 464 с
4. Chamberlin, Donald D; Boyce, Raymond F (1974). "SEQUEL: A Structured English Query Language" (PDF). Proceedings of the 1974 ACM SIGFIDET Workshop on Data Description, Access and Control. Association for Computing Machinery: 249–64.
5. «MySQL по максимуму» - Бэрн Шварц, Петр Зайцев, Вадим Ткаченко.
6. SQL Сборник рецептов» 2-е издание -Энтони Молинаро и Робер де Грааф
7. TIOBE Index [Электронный ресурс] : рейтинг популярности языков программирования // Официальный сайт. – URL: <https://www.tiobe.com/tiobe-index/> (дата обращения: 29.04.2025).
8. Браун И. Веб-разработка с применением Node и Express. Полноценное использование стека JavaScript. 2-е издание. — СПб.: Питер, 2021. — 336 с.: ил.
9. Хэррон Д. Node.js. Разработкасерверных веб-приложений в JavaScript: Пер. с англ. СлинкинаА. А. — М.: ДМК Пресс, 2012. — 144 с.: ил.
- 10.Осипов Д. Л. Технологии проектирования баз данных. — М.: ДМК Пресс, 2019. — 498 с.
- 11.Node.js API [Электронный ресурс] // Node.js: официальная документация. – URL: <https://nodejs.org/docs/latest/api> (дата обращения: 13.05.2025).
- 12.EJS [Электронный ресурс]. – URL: <https://ejs.co> (дата обращения: 23.05.2025).
- 13.MySQL2 Documentation [Электронный ресурс] : официальная документация библиотеки Node.js. – URL: <https://sidorares.github.io/node-mysql2/docs> (дата обращения: 23.05.2025).

14. Multer [Электронный ресурс] : middleware для обработки multipart/form-data в Node.js // GitHub : официальный репозиторий. - URL: <https://github.com/expressjs/multer> (дата обращения: 23.05.2025).
15. JWT Introduction [Электронный ресурс]. - URL: <https://jwt.io/introduction> (дата обращения: 23.05.2025).
16. jsonwebtoken [Электронный ресурс] : библиотека для работы с JWT в Node.js // GitHub : официальный репозиторий. - URL: <https://github.com/auth0/node-jsonwebtoken> (дата обращения: 25.05.2025).
17. Nodemailer [Электронный ресурс] : модуль для отправки электронной почты в Node.js // Официальный сайт. – URL: <https://nodemailer.com> (дата обращения: 23.05.2025).

## ПРИЛОЖЕНИЕ

Приложение А - Пожелания к системе автоматизированного хранения научных работ

1. Зав кафедры:

1.1. Хотелось бы, чтобы высылались оповещения на почту о том, что что-то надо загрузить или о том, что что-то было загружено, чтобы постоянно не проверять через сам репозиторий.

1.2. Хотелось бы иметь возможность отслеживать прогресс студента, чтобы знать на каком этапе работы он находится.

1.3. Хотелось бы иметь счетчик студентов, загрузивших работу в целом, а также у конкретного научного руководителя, чтобы знать прогресс у студентов, а также знать у какого научного руководителя отстаёт прогресс.

1.4. Хотелось бы иметь возможность выделить наиболее интересные работы, чтобы они выставлялись на обозрение, с целью демонстрации заказчикам.

1.5. Хотелось бы иметь возможность добавить работу в избранное, чтобы было проще найти её в будущем.

1.6. Хотелось бы, чтобы рядом с оценкой научного руководителя ставилась оценка антиплагиата, чтобы оценить то, как руководитель оценивает работы.

2. Норма контроль:

2.1. Хотел бы иметь возможность ставить отметку на пояснительной записке, чтобы можно было отправить её на доработку.

2.2. Хотелось бы иметь возможность оставлять замечания в пояснительной записке, чтобы студенты знали, что не так в выбранном месте.

2.3. Хотелось бы иметь возможность оставлять замечания для всей пояснительной записке, чтобы студенты знали, что не так с их работой.

2.4. Хотел бы видеть историю замечаний, чтобы проверять, что менялось в работе.

2.5. Хотелось бы иметь возможность диалога со студентом, чтобы иметь обратную связь по замечаниям к работе.

2.6. Хотелось бы, чтобы система имела настраиваемые привилегии для пользователей, чтобы разграничивать доступ к работам, например, по возможностям загрузки, просмотра или загрузки работ.

2.7. Хотелось бы иметь чек-лист, список которого бы содержал необходимые критерии, которые должна пройти пояснительная записка во время проверки у норма-контролёра, чтобы эти критерии были перед глазами во время проверки.

2.8. Хотелось бы, чтобы система могла автоматически проверять пояснительную записку на соответствие ГОСТам, чтобы самому не искать несоответствия.

2.9. Хотелось бы, чтобы в системе была кнопка, которая могла бы привести пояснительную записку в соответствии с ГОСТами, чтобы не экономить время на правках.

### 3. Хранитель:

3.1. Хотелось бы иметь возможность выдавать и менять привилегии для пользователей, чтобы разграничить доступ к выпускным работам. (Пересекается с 2.6)

### 4. Преподаватели:

4.1. Хотелось бы иметь возможность загружать ВКР, чтобы это могли делать не только студенты или ответственные за это лица.

4.2. Хотелось бы иметь возможность просматривать то, что было загружено, чтобы знать кто что сделал.

4.3. Хотелось бы, чтобы в системе можно было добавить демонстрацию к ВКР в виде презентации или видео, чтобы в будущем не просматривать всю работу, чтобы понять, о чем она.



4.4. Хотелось бы, чтобы в системе отображались сроки контроля работ, которые проходят студенты, чтобы знать, что и когда надо сдать.

4.5. Хотелось бы, чтобы система отправляла на почту оповещения, чтобы постоянно не проверять систему.

4.6. Хотелось бы, чтобы в системе имелся календарь со сроками контроля и сдачи работ, чтобы не искать в других источниках информации об сроках.

4.7. Хотелось бы, чтобы у выпускных работ отображались оценки, полученные за их защиту, чтобы не искать эти оценки в других источниках.

4.8. Хотелось бы, чтобы у выпускных работ отображались вопросы комиссии, что были на защите, чтобы знать, о чем спрашивали, если не присутствовал во время защиты.

4.9. Хотелось бы, чтобы отображались замечания с предзащиты чтобы знать, о замечаниях, если не присутствовал во время защиты.

4.10. Хотелось бы, иметь возможность оставлять замечания к выпускным работам, а также получать на замечания ответы, чтобы не переговаривать эту информацию устно или через способы источники связи. (Пересекается с 2.2, 2.3, 2.5).

4.11. Хотелось бы, иметь возможность составлять и изменять студентам список задач для доработки или исправлений, чтобы не обговаривать это устно или через другие способы связи.

4.12. Хотелось бы, чтобы при нажатии на задачу из списка (см. 4.10), автоматически переносила на участок пояснительной записки или кода, чтобы самому не искать, где находится это место.

4.13. Хотелось бы, чтобы было возможность просматривать историю изменения работ, чтобы знать, как изменялась работа. (Пересекается с 2.4.)

4.14. Хотелось бы, чтобы студентам приходили оповещения об сроках сдачи контроля работ, чтобы не оповещать их самолично или они не искали, где это можно узнать.

4.15. Хотелось бы иметь возможность показать работы будущим студентам, потенциальным заказчикам и т.д., чтобы поделиться информацией или продемонстрировать возможности кафедры.

4.16. Хотелось бы, чтобы имелась визуализация работы для тех, кто не хочет читать пояснительную записку (пересекается с 4.3.).

4.17. Хотелось бы, чтобы между работами можно было бы устанавливать связи, чтобы отслеживать изменения, которые претерпевает разработка, начатая в первой работе или объединять эти работы в какой-то общей темой.

4.18. Хотелось бы, чтобы к работам можно было бы ставить теги, чтобы упростить поиск и сортировку работ.

4.19. Хотелось бы, чтобы история исправлений была на человеко-читаемом языке, чтобы было понятно, что менялось.

4.20. Хотелось бы, чтобы система имела возможность быстрой интеграции каких-то новых этапов контроля ВКР, чтобы не это можно было сделать без трудностей для администратора (например).

## 5. Студенты:

5.1. Хотелось бы, чтобы в системе имелась возможность провести ревью выпускной работы, чтобы научный руководитель (или другой проверяющий) смог оценить изменения перед загрузкой работы в репозиторий. (Пересекается с 2.2, 2.3, 4.10)

5.2. Хотелось бы, чтобы в системе имелся эмулятор, который бы позволял смотреть работу кода.

## Приложение Б - Листинг кода приложения

```
const express = require('express');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const mysql = require('mysql2/promise');
const config = require('./config');
const cookieParser = require('cookie-parser');
const multer = require('multer');
const path = require('path');
const session = require('express-session');
const csrf = require('csurf');
const csrfProtection = csrf({ cookie: true });
const fs = require('node:fs')

const app = express();
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(cookieParser());
app.set('view engine', 'ejs');
app.use((req, res, next) => {
  res.locals.getStatusText = getStatusText;
  next();
});
app.use(express.static('public'));
app.use('/data', express.static('data'));
app.use(session({
  secret: process.env.SESSION_SECRET || '57A456BAEFAE429CA0F82D5E1D5EC537 ',
  resave: false,
  saveUninitialized: false,
  cookie: { secure: process.env.NODE_ENV === 'production' }
}));

// Подключение к MySQL (XAMPP)
const pool = mysql.createPool({
  host: config.DB_HOST,
  user: config.DB_USER,
  password: config.DB_PASSWORD,
  database: config.DB_NAME,
  waitForConnections: true,
  connectionLimit: 10
});

const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, 'uploads/');
  },
  filename: (req, file, cb) => {
    // Сохраняем оригинальное имя файла с правильной кодировкой
    const originalName = Buffer.from(file.originalname,
'latin1').toString('utf8');
    const ext = path.extname(originalName);
    const baseName = path.basename(originalName, ext);
    const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1E9);
    cb(null, `${baseName}-${uniqueSuffix}${ext}`);
  }
});

const upload = multer({
  storage: storage,
  fileFilter: (req, file, cb) => {
    const allowedTypes = [
      'application/pdf',

```

```

        'application/msword',
        'application/vnd.openxmlformats-officedocument.wordprocessingml.document'
    ];
    if (allowedTypes.includes(file.mimetype)) {
        cb(null, true);
    } else {
        cb(new Error('Разрешены только PDF и DOCX'), false);
    }
}
});

const documentStorage = multer.diskStorage({
    destination: (req, file, cb) => {
        cb(null, 'uploads/documents/');
    },
    filename: (req, file, cb) => {
        const originalName = Buffer.from(file.originalname,
'latin1').toString('utf8');
        const ext = path.extname(originalName);
        const baseName = path.basename(originalName, ext);
        const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1E9);
        cb(null, `${baseName}-${uniqueSuffix}${ext}`);
    }
});

const archiveStorage = multer.diskStorage({
    destination: (req, file, cb) => {
        cb(null, 'uploads/archives/');
    },
    filename: (req, file, cb) => {
        const originalName = Buffer.from(file.originalname,
'latin1').toString('utf8');
        const ext = path.extname(originalName);
        const baseName = path.basename(originalName, ext);
        const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1E9);
        cb(null, `${baseName}-${uniqueSuffix}${ext}`);
    }
});

// Фильтры для разных типов файлов
const documentFilter = (req, file, cb) => {
    const allowedTypes = [
        'application/pdf',
        'application/msword',
        'application/vnd.openxmlformats-officedocument.wordprocessingml.document'
    ];
    if (allowedTypes.includes(file.mimetype)) {
        cb(null, true);
    } else {
        cb(new Error('Разрешены только PDF и DOCX'), false);
    }
};

const archiveFilter = (req, file, cb) => {
    const allowedTypes = [
        'application/zip',
        'application/x-zip-compressed',
        'application/x-rar-compressed',
        'application/octet-stream'
    ];
    if (allowedTypes.includes(file.mimetype)) {
        cb(null, true);
    } else {

```

```

        cb(new Error('Разрешены только ZIP и RAR архивы'), false);
    }
};

// Создаем экземпляры Multer
const uploadDocument = multer({
    storage: documentStorage,
    fileFilter: documentFilter
});

const uploadArchive = multer({
    storage: archiveStorage,
    fileFilter: archiveFilter
});

//Автоматический перевод на логин или в ЛК в зависимости от авторизации
app.get('/', async (req, res) => {
    try {
        // 1. Проверяем JWT из куков
        const token = req.cookies?.jwt_token;

        if (!token) {
            return res.redirect('/login'); // Не авторизован -> на вход
        }

        // 2. Верифицируем токен
        const decoded = jwt.verify(token, process.env.JWT_SECRET);

        // 3. Определяем главную роль
        const mainRole = getMainRole(decoded.roles);

        // 4. Редирект в личный кабинет
        res.redirect(`/${mainRole}/dashboard`);

    } catch (error) {
        // Если токен невалидный
        res.clearCookie('jwt_token');
        res.redirect('/login');
    }
});

// Роут для формы входа (GET)
app.get('/login', (req, res) => {
    res.render('login', { error: null }); // Форма из login.ejs
});

// Авторизация (POST)
app.post('/login', async (req, res) => {
    //Получение логина и пароля из формы
    const { login, password } = req.body;

    try {
        // 1. Поиск пользователя
        const [users] = await pool.query('SELECT * FROM users WHERE email = ?',
[login]); //Переменная users хранит результат запроса из pool'a
        if (users.length === 0) {
            return res.status(401).render('login', { error: 'Неверный логин или
пароль' });
        }
        const user = users[0];
        // 2. Проверка пароля
        if (password !== user.password_hash) {
            return res.status(401).render('login', { error: 'Неверный логин или
пароль' });
        }
    }
});

```

```

    }
    // 3. Получаем роли пользователя
    const [roles] = await pool.query(`
      SELECT r.name
      FROM user_roles ur
      JOIN roles r ON ur.role_id = r.id
      WHERE ur.user_id = ?
    `, [user.id]);

    // 4. Определяем главную роль (для редиректа)
    const userRoles = roles.map(role => role.name);
    const mainRole = getMainRole(roles.map(r => r.name));

    // 5. Создаем JWT токен с информацией о ролях
    const token = jwt.sign(
      {
        id: user.id,
        email: user.email,
        roles: userRoles
      },
      process.env.JWT_SECRET,
      { expiresIn: '8h' }
    );

    // 6. Установка куки и редирект
    res.cookie('jwt_token', token, {
      httpOnly: true,
      maxAge: 8 * 60 * 60 * 1000 // 8 часов
    });
    // Редирект в личный кабинет согласно роли
    console.log(`Пользователь ${login} успешно вошел в систему как ${mainRole}`);
    return res.redirect(`${mainRole}/dashboard`)

  } catch (error) {
    console.error('Ошибка при авторизации:', error);
    return res.status(500).render('error', {
      message: 'Внутренняя ошибка сервера',
      error: error.message
    });
  }
});

// Выход
app.get('/logout', (req, res) => {
  res.clearCookie('jwt_token');
  res.redirect('/login');
});

//
// Студенческая часть
//

// Кабинет студента
app.get('/student/dashboard', authenticateToken, checkRole('student'), async (req, res) => {
  try {
    // 1. Получаем данные студента
    const studentData = await getStudentData(req.user.id)

    // 2. Проверяем, что данные найдены
    if (!studentData || studentData.length === 0) {

```

```

        throw new Error('Данные студента не найдены');
    }

    // 3. Получаем дипломные работы студента
    const [diplomas] = await pool.query(`
        SELECT
            id,
            title,
            status,
            created_at,
            updated_at
        FROM diplomas
        WHERE student_id = ?
        ORDER BY updated_at DESC
    `, [req.user.id]);

    // 4. Рендерим шаблон с обязательными данными
    res.render('student/dashboard', {
        student: studentData, // Важно: передаем первый элемент массива
        diplomas: diplomas || [], // На случай если работ нет
        user: req.user // Передаем также данные авторизованного пользователя
    });

    } catch (error) {
        console.error('Ошибка загрузки кабинета:', error);
        res.status(500).render('error', {
            message: 'Ошибка загрузки личного кабинета',
            error: error.message
        });
    }
});

// Просмотр работы и обсуждений
app.get('/student/diploma/:id', authenticateToken, csrfProtection,
checkRole('student'), async (req, res) => {
    try {
        const diplomaId = req.params.id;
        const studentData = await getStudentData(req.user.id)
        const [diploma] = await pool.query(`
            SELECT d.*, u.first_name AS supervisor_name
            FROM diplomas d
            JOIN users u ON d.supervisor_id = u.id
            WHERE d.id = ? AND d.student_id = ?
        `, [diplomaId, req.user.id]);

        const [documents] = await pool.query(`
            SELECT * FROM documents
            WHERE diploma_id = ?
            ORDER BY uploaded_at DESC
        `, [diplomaId]);
        /*
        const [discussions] = await pool.query(`
            SELECT d.*, u.first_name AS author_name
            FROM discussions d
            JOIN users u ON d.created_by = u.id
            WHERE d.diploma_id = ?
            ORDER BY d.created_at DESC
        `, [diplomaId]);
        */

        res.render('student/diploma', {
            diploma: diploma[0],
            student: studentData,

```

```

        documents,
        //discussions,
        user: req.user,
        csrfToken: req.csrfToken()
    });
} catch (error) {
    console.error(error);
    res.status(500).send('Ошибка сервера');
}
});

// Показ формы для создания новой работы
app.get('/student/new-diploma', authenticateToken, checkRole('student'),
async (req, res) => {
    try {
        const supervisors = await getSupervisors();
        res.render('student/new-diploma', {
            supervisors,
            user: req.user,
            values: {} // Добавляем пустой объект values
        });
    } catch (error) {
        console.error('Ошибка при загрузке формы:', error);
        res.status(500).render('error', {
            message: 'Ошибка загрузки формы',
            error: error.message
        });
    }
});

// Обработка отправки формы
app.post('/student/new-diploma', authenticateToken, checkRole('student'),
async (req, res) => {
    const { title, description, supervisor_id } = req.body;

    try {
        // Валидация данных
        if (!title || !supervisor_id) {
            const supervisors = await getSupervisors();
            return res.render('student/new-diploma', {
                error: 'Название и научный руководитель обязательны',
                supervisors,
                values: req.body, // Передаем введенные значения обратно в форму
                user: req.user
            });
        }
        // Создание новой работы
        const [result] = await pool.query(`
            INSERT INTO diplomas (student_id, supervisor_id, title, description,
status)
VALUES (?, ?, ?, ?, 'draft')
`, [req.user.id, supervisor_id, title, description || null]);

        const diplomaId = result.insertId; //Получение ИД только что созданной
работы
        // Создание обсуждения с руководителем
        const supDiscussionId = await createDiscussion(
            diplomaId,
            `Обсуждение работы "${title}" с научным руководителем`
        );
        // Добавляем участников (студент и научный руководитель)
        await addUserToDiscussion(supDiscussionId, req.user.id);
        await addUserToDiscussion(supDiscussionId, supervisor_id);
    }
});

```



```

// Поиск нормоконтроля для вставки
const [normControl] = await pool.query(`
  SELECT user_id FROM user_roles WHERE role_id = (
    SELECT id FROM roles WHERE name = 'norm_control'
  ) LIMIT 1
`);

//Если нормоконтроль найден
if (normControl.length > 0) {
  const normControlUserId = normControl[0].user_id;
  //Создать обсуждение
  const normControlDiscussionId = await createDiscussion(
    diplomaId,
    `Обсуждение оформления работы "${title}" с нормоконтролем`
  );

  //Вставка пользователей
  await addUserToDiscussion(normControlDiscussionId, req.user.id);
  await addUserToDiscussion(normControlDiscussionId, normControlUserId);
}
res.redirect('/student/dashboard');
} catch (error) {
  //Если ошибка, откат транзакции
  await conn.rollback();
  console.error('Ошибка при создании работы:', error);
  res.status(500).render('student/new-diploma', {
    error: 'Ошибка при создании работы',
    values: req.body,
    user: req.user
  });
}
});

//Просмотр обсуждений
app.get('/student/discussions', authenticateToken, checkRole('student'),
async (req, res) => {
  try {
    //Получаем данные студента
    const studentData = await getStudentData(req.user.id)
    // Получаем все обсуждения, где участвует студент
    const [discussions] = await pool.query(`
      SELECT
        d.id,
        d.title,
        d.created_at,
        dp.title AS diploma_title,
        dp.status AS diploma_status,
        GROUP_CONCAT(
          DISTINCT CONCAT(u.first_name, ' ', u.last_name)
          SEPARATOR ', '
        ) AS participants
      FROM discussions d
      JOIN discussion_user du ON d.id = du.discussion_id
      JOIN diplomas dp ON d.diploma_id = dp.id
      JOIN users u ON du.user_id = u.id
      WHERE d.id IN (
        SELECT discussion_id
        FROM discussion_user
        WHERE user_id = ?
      )
      GROUP BY d.id
      ORDER BY d.created_at DESC
    `, [req.user.id]);
  }

```

```

// Получаем последние сообщения для каждого обсуждения
const discussionsWithLastMessage = await Promise.all(
  discussions.map(async discussion => {
    const [lastMessage] = await pool.query(`
      SELECT m.content, m.created_at,
             CONCAT(u.first_name, ' ', u.last_name) AS author_name
      FROM messages m
      JOIN users u ON m.sender_id = u.id
      WHERE m.discussion_id = ?
      ORDER BY m.created_at DESC
      LIMIT 1
    `, [discussion.id]);

    return {
      ...discussion,
      lastMessage: lastMessage[0] || null,
      unreadCount: 0 // Можно добавить логику подсчета непрочитанных
    };
  })
);

res.render('student/discussions', {
  student: studentData,
  discussions: discussionsWithLastMessage,
  getStatusText // Функция для отображения статуса
});
} catch (error) {
  console.error('Ошибка загрузки обсуждений:', error);
  res.status(500).render('error', {
    message: 'Ошибка загрузки списка обсуждений',
    error: error.message
  });
}
});

// Страница создания заявки на проверку
app.get('/student/check', authenticateToken, checkRole('student'), async
(req, res) => {
  try {
    // Получаем работы студента
    const [diplomas] = await pool.query(`
      SELECT id, title FROM diplomas
      WHERE student_id = ? AND status != 'defended'
    `, [req.user.id]);

    // Получаем возможных проверяющих (научные руководители и нормоконтроль)
    const [reviewers] = await pool.query(`
      SELECT u.id, u.first_name, u.last_name, r.name as role
      FROM users u
      JOIN user_roles ur ON u.id = ur.user_id
      JOIN roles r ON ur.role_id = r.id
      WHERE r.name IN ('supervisor', 'norm_control')
    `);

    // Получаем файлы студента
    const [documents] = await pool.query(`
      SELECT id, file_name FROM documents
      WHERE user_id = ? AND diploma_id IN (
        SELECT id FROM diplomas WHERE student_id = ?
      )
    `, [req.user.id, req.user.id]);

    const [files] = await pool.query(`
      SELECT id, file_name FROM files
      WHERE user_id = ? AND diploma_id IN (
        SELECT id FROM diplomas WHERE student_id = ?
    `);
  }
});

```

```

    )
    `, [req.user.id, req.user.id]);

    res.render('student/create-check', {
        user: req.user,
        diplomas,
        reviewers,
        documents,
        files,
        csrfToken: req.csrfToken()
    });
} catch (error) {
    console.error('Ошибка загрузки формы проверки:', error);
    res.status(500).render('error', {
        message: 'Ошибка загрузки формы',
        error: error.message
    });
}
});

// Обработка отправки заявки на проверку
app.post('/student/check', authenticateToken, checkRole('student'), async
(req, res) => {
    const { type, diploma_id, reviewer_id, student_comment, document_ids,
file_ids } = req.body;

    try {
        // Валидация
        if (!diploma_id || !reviewer_id || (!document_ids && !file_ids)) {
            return res.status(400).json({
                error: 'Заполните обязательные поля: Работа, Проверяющий и Файлы'
            });
        }

        const conn = await pool.getConnection();
        await conn.beginTransaction();

        try {
            // Проверяем, что работа принадлежит студенту
            const [diploma] = await conn.query(`
                SELECT id FROM diplomas
                WHERE id = ? AND student_id = ?
            `, [diploma_id, req.user.id]);

            if (!diploma.length) {
                throw new Error('Работа не найдена или нет доступа');
            }

            // Определяем тип проверки
            const isNormControl = type === 'norm_control';
            const tableName = isNormControl ? 'norm_control_checks' : 'reviews';

            // Создаем запись о проверке
            const [check] = await conn.query(`
                INSERT INTO ${tableName} (
                    diploma_id,
                    ${isNormControl ? 'inspector_id' : 'reviewer_id'},
                    student_comment,
                    status
                ) VALUES (?, ?, ?, 'pending')
            `, [diploma_id, reviewer_id, student_comment]);

            const checkId = check.insertId;

```

```

    // Прикрепляем документы
    if (document_ids) {
        const docIds = Array.isArray(document_ids) ? document_ids :
[document_ids];
        await Promise.all(docIds.map(async docId => {
            await conn.query(`
                INSERT INTO review_documents (review_id, document_id)
                VALUES (?, ?)
            `, [checkId, docId]);
        }));
    }

    // Прикрепляем файлы
    if (file_ids) {
        const fIds = Array.isArray(file_ids) ? file_ids : [file_ids];
        await Promise.all(fIds.map(async fileId => {
            await conn.query(`
                INSERT INTO review_files (review_id, file_id)
                VALUES (?, ?)
            `, [checkId, fileId]);
        }));
    }

    // Обновляем статус работы
    await conn.query(`
        UPDATE diplomas SET status = ?
        WHERE id = ?
    `, [isNormControl ? 'in_normcontrol' : 'in_review', diploma_id]);

    await conn.commit();

    res.json({
        success: true,
        redirect: `/student/diploma/${diploma_id}`
    });
} catch (error) {
    await conn.rollback();
    throw error;
} finally {
    conn.release();
}
} catch (error) {
    console.error('Ошибка создания проверки:', error);
    res.status(500).json({
        error: 'Не удалось создать проверку: ' + error.message
    });
}
});

//
// Научный руководитель
//

// Кабинет руководителя
app.get('/supervisor/dashboard', authenticateToken, checkRole('supervisor'),
async (req, res) => {
    try {
        const supervisor = await getSupervisorData(req.user.id);
        const students = await getStudentsByYear(req.user.id);

        // Получаем уникальные года из результатов
        const years = [...new Set(students.map(s => s.year))].sort((a,b) => b -
a);

```

```

    res.render('supervisor/dashboard', {
      supervisor,
      students,
      years,
      currentYear: years.length > 0 ? years[0] : new Date().getFullYear()
    });
  } catch (error) {
    console.error('Ошибка загрузки кабинета:', error);
    res.status(500).render('error');
  }
});

// Страница студента
app.get('/supervisor/student/:id', authenticateToken,
checkRole('supervisor'), async (req, res) => {
  const studentId = req.params.id;

  const [student] = await pool.query(`
    SELECT u.*, s.group_id, sg.name as group_name
    FROM users u
    LEFT JOIN students s ON u.id = s.user_id
    LEFT JOIN study_groups sg ON s.group_id = sg.id
    WHERE u.id = ?
  `, [studentId]);

  const [diplomas] = await pool.query(`
    SELECT * FROM diplomas
    WHERE student_id = ?
  `, [studentId]);

  console.log('Supervisor ID:', req.user.id, 'Student:', student[0]);

  // Получаем документы для всех дипломов
  const diplomasWithDocs = await Promise.all(diplomas.map(async diploma => {
    const [docs] = await pool.query('SELECT * FROM documents WHERE diploma_id
= ?', [diploma.id]);
    return { ...diploma, documents: docs };
  }));

  res.render('supervisor/student', {
    student: student[0],
    diplomas: diplomasWithDocs,
    supervisorId: req.user.id
  });
});

//
//
//

//
//Заведующий кафедрой
//

//Кибнет заведующего кафедрой
app.get('/head_of_department/dashboard', authenticateToken,
checkRole('head_of_department'), async (req, res) => {
  try {
    const {view_mode = 'by_groups'} = req.query;
    //Набор руководителей со студентами за этот год
    const [supervisors] = await pool.query(`
      SELECT u.id,
        CONCAT(u.last_name, ' ', u.first_name, ' ',
        COALESCE(u.patronymic, '')) AS full_name,

```

```

        COUNT(DISTINCT d.student_id) AS total_students,
        SUM(CASE WHEN d.status = 'draft' THEN 1 ELSE 0 END) AS
draft_count,
        SUM(CASE WHEN d.status = 'in_review' THEN 1 ELSE 0 END) AS
in_review_count,
        SUM(CASE WHEN d.status = 'approved' THEN 1 ELSE 0 END) AS
approved_count,
        SUM(CASE WHEN d.status = 'defended' THEN 1 ELSE 0 END) AS
defended_count
    FROM
        users u
    JOIN
        supervisors s ON u.id = s.user_id
    LEFT JOIN
        diplomas d ON u.id = d.supervisor_id
    `);
//Набор групп со студентами за этот год
const [groups] = await pool.query(`
SELECT g.id, g.name,
        COUNT(DISTINCT d.student_id) AS total_students,
        SUM(CASE WHEN d.status = 'draft' THEN 1 ELSE 0 END) AS
draft_count,
        SUM(CASE WHEN d.status = 'in_review' THEN 1 ELSE 0 END) AS
in_review_count,
        SUM(CASE WHEN d.status = 'approved' THEN 1 ELSE 0 END) AS
approved_count,
        SUM(CASE WHEN d.status = 'defended' THEN 1 ELSE 0 END) AS
defended_count
    FROM
        study_groups g
    JOIN
        students s ON g.id = s.group_id
    LEFT JOIN
        diplomas d ON s.user_id = d.student_id;
    `);

const headOfDepartment = await getHeadDepData(req.user.id);
res.render('head_of_department/dashboard', {
    user: headOfDepartment,
    supervisors,
    groups,
    view_mode
});
} catch (error) {
    console.error('Ошибка загрузки кабинета:', error);
    res.status(500).render('error', {
        message: 'Ошибка загрузки личного кабинета',
        error: error.message
    });
}
});

//Список годов
app.get('/head_of_department/years', authenticateToken,
checkRole('head_of_department'), async (req, res) =>{
    try{
        const headOfDepartment = await getHeadDepData(req.user.id);

        const [years] = await pool.query(`
            WITH years AS (
                SELECT DISTINCT YEAR(created_at) AS year FROM diplomas
            )

            SELECT

```

```

        y.year,
        COUNT(DISTINCT d.student_id) AS students_count,
        COUNT(DISTINCT d.supervisor_id) AS supervisors_count,
        COUNT(DISTINCT fg.diploma_id) AS defended_count
    FROM
        years y
    LEFT JOIN
        diplomas d ON YEAR(d.created_at) = y.year
    LEFT JOIN
        final_grades fg ON YEAR(fg.defended_at) = y.year
    GROUP BY
        y.year
    ORDER BY
        y.year DESC;
`);

res.render('head_of_department/years', {
    user: headOfDepartment,
    years
});
} catch (error) {
    console.error('Ошибка загрузки списка годов заведующего кафедрой:',
error);
    res.status(500).render('error', {
        message: 'Ошибка загрузки списка учебных годов',
        error: error.message
    });
}
});

//Страница статистики за конкретный год
app.get('/head_of_department/year/:num',
authenticateToken,checkRole('head_of_department'), async (req, res) =>{
    const numOfYear = req.params.num;
    try{
        const headOfDepartment = await getHeadDepData(req.user.id);
        const {view_mode = 'by_groups'} = req.query;
        //Набор руководителей со студентами за этот год
        const [supervisors] = await pool.query(`
            SELECT u.id,
                CONCAT(u.last_name, ' ', u.first_name, ' ', COALESCE(u.patronymic,
'')) AS full_name,
                COUNT(DISTINCT d.student_id) AS total_students,
                SUM(CASE WHEN d.status = 'draft' THEN 1 ELSE 0 END) AS draft_count,
                SUM(CASE WHEN d.status = 'in_review' THEN 1 ELSE 0 END) AS
in_review_count,
                SUM(CASE WHEN d.status = 'approved' THEN 1 ELSE 0 END) AS
approved_count,
                SUM(CASE WHEN d.status = 'defended' THEN 1 ELSE 0 END) AS
defended_count
            FROM
                users u
            JOIN
                supervisors s ON u.id = s.user_id
            LEFT JOIN
                diplomas d ON u.id = d.supervisor_id
            WHERE YEAR(d.created_at) = ?
            `, [numOfYear]);
        //Набор групп со студентами за этот год
        const [groups] = await pool.query(`
            SELECT g.id, g.name,
                COUNT(DISTINCT d.student_id) AS total_students,
                SUM(CASE WHEN d.status = 'draft' THEN 1 ELSE 0 END) AS draft_count,

```

```

        SUM(CASE WHEN d.status = 'in_review' THEN 1 ELSE 0 END) AS
in_review_count,
        SUM(CASE WHEN d.status = 'approved' THEN 1 ELSE 0 END) AS
approved_count,
        SUM(CASE WHEN d.status = 'defended' THEN 1 ELSE 0 END) AS
defended_count
FROM
    study_groups g
JOIN
    students s ON g.id = s.group_id
LEFT JOIN
    diplomas d ON s.user_id = d.student_id
WHERE YEAR(d.created_at) = ?
`, [numOfYear]);
res.render('head_of_department/year', {
    user: headOfDepartment,
    supervisors,
    groups,
    view_mode,
    numOfYear
})
} catch (error) {
    console.error('Ошибка загрузки информации за год:', error);
    res.status(500).render('error', {
        message: 'Ошибка загрузки страницы',
        error: error.message
    });
}
});

//Страница работ конкретного руководителя
app.get('/head_of_department/supervisor/:id', authenticateToken,
checkRole('head_of_department'), async (req, res) =>{

    try {
        const headOfDepartment = await getHeadDepData(req.user.id);
        const supervisorId = req.params.id;

        const supervisor = await getSupervisorData(supervisorId);
        const students = await getStudentsByYear(supervisorId, req.user.id);

        const years = [...new Set(students.map(s => s.year))].sort((a,b) => b -
a);
        res.render('head_of_department/supervisor', {
            user: headOfDepartment,
            supervisor,
            students,
            years,
            currentYear: years.length > 0 ? years[0] : new Date().getFullYear()
        });

    } catch (error) {
        console.error('Ошибка загрузки страницы руководителя:', error);
        res.status(500).render('error', {
            message: 'Ошибка загрузки страницы',
            error: error.message
        });
    }

});

app.get('/head_of_department/favorites', authenticateToken,
checkRole('head_of_department'), async (req, res) => {
    try{

```



```

const headOfDepartment = await getHeadDepData(req.user.id);
const [works] = await pool.query(`
  SELECT
    CONCAT(u.last_name, ' ', u.first_name, ' ', COALESCE(u.patronymic,
'')) AS student_name,
    d.title AS diploma_title,
    CONCAT(sup.last_name, ' ', sup.first_name, ' ',
COALESCE(sup.patronymic, '')) AS supervisor_name,
    d.status AS diploma_status,
    YEAR(d.created_at) AS year
  FROM
    favorites f
  JOIN
    diplomas d ON f.diploma_id = d.id
  JOIN
    users u ON d.student_id = u.id
  JOIN
    users sup ON d.supervisor_id = sup.id
  WHERE
    f.user_id = ?
`, [req.user.id])
const years = [...new Set(works.map(w => w.year))].sort((a,b) => b - a);
res.render('head_of_department/favorites', {
  user: headOfDepartment,
  works,
  years,
  currentYear: years.length > 0 ? years[0] : new Date().getFullYear()
});
} catch (error){
  console.error('Ошибка загрузки избранного:', error);
  res.status(500).render('error', {
    message: 'Ошибка загрузки страницы',
    error: error.message
  });
}
});

// Список студентов
app.get('/head_of_department/students', authenticateToken,
checkRole('head_of_department'), async (req, res) => {
  try {
    const [students] = await pool.query(`
      SELECT
        u.id, u.first_name, u.last_name, u.patronymic,
        sg.name as group_name,
        d.title as diploma_title,
        d.status as diploma_status,
        CONCAT(sup.first_name, ' ', sup.last_name) as supervisor_name
      FROM users u
      JOIN students s ON u.id = s.user_id
      JOIN study_groups sg ON s.group_id = sg.id
      LEFT JOIN diplomas d ON u.id = d.student_id
      LEFT JOIN users sup ON d.supervisor_id = sup.id
      ORDER BY u.last_name, u.first_name
    `);

    res.render('head_of_department/students', {
      user: req.user,
      students
    });
  } catch (error) {
    console.error('Ошибка загрузки списка студентов:', error);
    res.status(500).render('error');
  }
}

```

```

});

// Список руководителей
app.get('/head_of_department/supervisors', authenticateToken,
checkRole('head_of_department'), async (req, res) => {
  try {
    const [supervisors] = await pool.query(`
      SELECT
        u.id, u.first_name, u.last_name, u.patronymic,
        s.academic_degree, s.position,
        COUNT(d.id) as students_count
      FROM users u
      JOIN supervisors s ON u.id = s.user_id
      LEFT JOIN diplomas d ON u.id = d.supervisor_id
      GROUP BY u.id
      ORDER BY u.last_name, u.first_name
    `);

    res.render('head_of_department/supervisors', {
      user: req.user,
      supervisors
    });
  } catch (error) {
    console.error('Ошибка загрузки списка руководителей:', error);
    res.status(500).render('error');
  }
});

// Статистика
app.get('/head_of_department/statistics', authenticateToken,
checkRole('head_of_department'), async (req, res) => {
  try {
    // Статистика по статусам работ
    const [statusStats] = await pool.query(`
      SELECT
        status,
        COUNT(*) as count
      FROM diplomas
      GROUP BY status
    `);

    // Статистика по группам
    const [groupStats] = await pool.query(`
      SELECT
        sg.name as group_name,
        COUNT(DISTINCT s.user_id) as students_count,
        COUNT(d.id) as diplomas_count
      FROM study_groups sg
      LEFT JOIN students s ON sg.id = s.group_id
      LEFT JOIN diplomas d ON s.user_id = d.student_id
      GROUP BY sg.id
    `);

    res.render('head_of_department/statistics', {
      user: req.user,
      statusStats,
      groupStats
    });
  } catch (error) {
    console.error('Ошибка загрузки статистики:', error);
    res.status(500).render('error');
  }
});

```

```

//Загрузка документа
app.post('/upload/document', authenticateToken,
checkRole('student','supervisor'), upload.single('document'), async (req,
res) => {
  try {
    if (!req.file) {
      return res.status(400).json({ error: 'Файл не был загружен' });
    }
    console.log('Download request:', {
      user: req.user.id,
      ip: req.ip
    });
    // Получаем оригинальное имя файла с правильной кодировкой
    const originalName = Buffer.from(req.file.originalname,
'latin1').toString('utf8');
    await pool.query(
      'INSERT INTO documents (diploma_id, user_id, file_path, file_name,
file_type) VALUES (?, ?, ?, ?, ?)',
      [req.body.diploma_id, req.user.id, req.file.path, originalName,
req.file.mimetype]
    );
    res.redirect(`/student/diploma/${req.body.diploma_id}`);
  } catch (error) {
    console.error('Ошибка загрузки файла:', error);
    res.status(500).json({ error: 'Ошибка загрузки файла' });
  }
}
);

//Удаление документа
app.post('/document/delete', authenticateToken, csrfProtection, async (req,
res) => {
  const { doc_id, diploma_id } = req.body;

  try {
    // 1. Проверяем права доступа
    const [access] = await pool.query(`
      SELECT
        d.user_id,
        dl.student_id,
        dl.supervisor_id,
        ur.role_id
      FROM documents d
      JOIN diplomas dl ON d.diploma_id = dl.id
      LEFT JOIN user_roles ur ON ur.user_id = ? AND ur.role_id = (
        SELECT id FROM roles WHERE name = 'administrator'
      )
      WHERE d.id = ?
    `, [req.user.id, doc_id]);

    if (!access[0]) {
      return
      res.status(404).redirect(`/student/diploma/${diploma_id}?error=Документ не
найден`);
    }

    // 2. Проверка прав (студент, руководитель или админ)
    const isOwner = access[0].user_id === req.user.id;
    const isStudent = access[0].student_id === req.user.id;
    const isSupervisor = access[0].supervisor_id === req.user.id;
    const isAdmin = !!access[0].role_id;

```

```

    if (!isOwner && !isStudent && !isSupervisor && !isAdmin) {
      return
    }
    res.status(403).redirect(`/student/diploma/${diploma_id}?error=Нет прав на
    удаление`);
  }

  // 3. Удаление файла и записи
  const [document] = await pool.query('SELECT file_path FROM documents
  WHERE id = ?', [doc_id]);

  if (document[0]?.file_path) {
    fs.unlink(document[0].file_path, (err) => {
      if (err) console.error('Ошибка удаления файла:', err);
    });
  }

  await pool.query('DELETE FROM documents WHERE id = ?', [doc_id]);

  res.redirect(`/student/diploma/${diploma_id}?success=Файл удален`);
} catch (error) {
  console.error('Ошибка удаления:', error);
  res.redirect(`/student/diploma/${diploma_id}?error=Ошибка сервера`);
}
});

//Скачивание документа
app.get('/download/:doc_id', authenticateToken, async (req, res) => {
  try {
    const { doc_id } = req.params;

    // 1. Поиск записи в таблице
    const [document] = await pool.query(`
    SELECT d.file_name, d.file_path, dl.student_id, dl.supervisor_id
    FROM documents d
    JOIN diplomas dl ON d.diploma_id = dl.id
    WHERE d.id = ?
    `, [doc_id]);

    if (!document.length) {
      return res.status(404).send('Документ не найден');
    }

    const doc = document[0];
    const isAllowed = req.user.id === doc.student_id ||
      req.user.id === doc.supervisor_id ||
      req.user.roles.includes('administrator');

    if (!isAllowed) {
      return res.status(403).send('Доступ запрещен');
    }

    // 2. Формируем путь к файлу
    const filePath = path.join(__dirname, doc.file_path);

    // 3. Проверяем существование файла
    if (!fs.existsSync(filePath)) {
      return res.status(404).send('Файл не найден на сервере');
    }

    // 4. Отправляем файл
    res.download(filePath, doc.file_name, (err) => {
      if (err) console.error('Ошибка скачивания:', {
        error: err,

```

```

        filePath: filePath,
        docId: doc_id
    });
});

} catch (error) {
    console.error('Download error:', {
        error: error,
        params: req.params,
        user: req.user
    });
    res.status(500).send('Ошибка сервера');
}
});

// Создание обсуждения
app.post('/student/new-discussion', authenticateToken, checkRole('student'),
async (req, res) => {
    const { diploma_id, title, content } = req.body;

    try {
        await pool.query(`
            INSERT INTO discussions (diploma_id, created_by, title, content)
            VALUES (?, ?, ?, ?)
        `, [diploma_id, req.user.id, title, content]);

        res.redirect(`/student/diploma/${diploma_id}`);
    } catch (error) {
        console.error(error);
        res.status(500).send('Ошибка создания обсуждения');
    }
});

// Проверка авторизации
function authenticateToken(req, res, next) {
    const token = req.cookies?.jwt_token;
    if (!token) {
        // Сохраняем URL для редиректа после входа
        req.session.returnTo = req.originalUrl;
        return res.redirect('/login');
    }
    jwt.verify(token, process.env.JWT_SECRET, (err, user) => {
        if (err) {
            res.clearCookie('jwt_token');
            return res.redirect('/login');
        }
        req.user = user;
        next();
    });
}

// Функция для определения главной роли
function getMainRole(userRoles) {
    const rolePriority = [
        'administrator',
        'head_of_department',
        'norm_control',
        'supervisor',
        'student'
    ];
    return rolePriority.find(role => userRoles.includes(role)) || 'student';
}

```

```

// Проверка конкретной роли
function checkRole(...roles) {
  return (req, res, next) => {
    if(!req.user?.roles){
      return res.status(500).render('error',{
        message: 'Доступ запрещен: нет укзаны роли'
      });
    }
    const hasRequiredRole = roles.some(role =>
req.user.roles.includes(role));
    if (!hasRequiredRole) {
      return res.status(403).render('error', {
        message: `Доступ запрещен: нет уровня допуска`
      });
    }
    next();
  };
}

// Функция для преобразования статуса в читаемый текст
function getStatusText(status) {
  const statusMap = {
    'draft': 'Черновик',
    'in_review': 'На проверке',
    'approved': 'Принято',
    'needs_correction': 'Требуется исправления'
  };
  return statusMap[status] || status;
}

// Вспомогательная функция для получения руководителей
async function getSupervisors() {
  const [supervisors] = await pool.query(`
    SELECT u.id, u.first_name, u.last_name, u.patronymic
    FROM users u
    JOIN supervisors s ON u.id = s.user_id
  `);
  return supervisors;
}

// Получение данных руководителя
async function getSupervisorData(userId) {
  const [data] = await pool.query(`
    SELECT u.*, s.academic_degree, s.position
    FROM users u
    JOIN supervisors s ON u.id = s.user_id
    WHERE u.id = ?
  `, [userId]);
  return data[0];
}

// Получение данных завкафедры
async function getHeadDepData(userId) {
  const [data] = await pool.query(`
    SELECT u.*
    FROM users u
    WHERE u.id = ?
  `, [userId]);
  return data[0];
}

// Получение данных студента
async function getStudentData(userId) {
  const [data] = await pool.query(`
    SELECT

```

```

        u.id,
        u.first_name,
        u.last_name,
        u.patronymic,
        sg.name AS group_name,
        s.student_card_number
    FROM users u
    JOIN students s ON u.id = s.user_id
    JOIN study_groups sg ON s.group_id = sg.id
    WHERE u.id = ?
`, [userId]);
return data[0];
}
// Получение студентов по году
async function getStudentsByYear(supervisorId, userID = supervisorId, year = null) {
    let sql = `
        SELECT
            u.id, u.first_name, u.last_name, u.patronymic,
            d.id as diploma_id, d.title, d.status,
            DATE_FORMAT(d.created_at, '%Y') as year,
            EXISTS(
                SELECT 1 FROM favorites f
                WHERE f.user_id = ? AND f.diploma_id = d.id
            ) AS is_favorite
        FROM diplomas d
        JOIN users u ON d.student_id = u.id
        WHERE d.supervisor_id = ?
    `;

    const params = [userID, supervisorId];

    if (year) {
        sql += ` AND YEAR(d.created_at) = ?`;
        params.push(year);
    }

    sql += ` ORDER BY u.last_name ASC`;

    const [students] = await pool.query(sql, params);
    return students;
}

//
// Обсуждения и сообщения
//

/**
 * Создает новое обсуждение для дипломной работы
 * @param {number} diploma_id - ID дипломной работы
 * @param {string} title - Заголовок обсуждения
 * @param {number} created_by - ID пользователя, создавшего обсуждение
 * @returns {Promise<number>} ID созданного обсуждения
 */

async function createDiscussion(diploma_id, title) {
    try {
        //Вставка в таблицу нового обсуждения
        const [result] = await pool.query(
            `INSERT INTO discussions (diploma_id, title)
            VALUES (?, ?)`,
            [diploma_id, title]
        );
    }
};

```

```

        return result.insertId; // Возвращаем ID созданного обсуждения
    } catch (error) {
        console.error('Ошибка при создании обсуждения:', error);
        throw error;
    }
}

/**
 * Добавляет пользователя в обсуждение
 * @param {number} discussion_id - ID обсуждения
 * @param {number} user_id - ID пользователя
 * @returns {Promise<void>}
 */

async function addUserToDiscussion(discussion_id, user_id) {
    try {
        //Запрос на вставку нового пользователя
        await pool.query(
            `INSERT INTO discussion_user (discussion_id, user_id)
            VALUES (?, ?)`,
            [discussion_id, user_id]
        );
    } catch (error) {
        console.error('Ошибка при добавлении пользователя в обсуждение:', error);
        throw error;
    }
}

// API для получения студентов
app.get('/api/supervisor/students', authenticateToken,
checkRole('supervisor'), async (req, res) => {
    const { year, query } = req.query;

    let sql = `
        SELECT u.id, u.first_name, u.last_name, d.title, d.status
        FROM diplomas d
        JOIN users u ON d.student_id = u.id
        WHERE d.supervisor_id = ? AND YEAR(d.created_at) = ?
    `;

    const params = [req.user.id, year];

    if (query) {
        sql += ` AND (u.first_name LIKE ? OR u.last_name LIKE ?)`;
        params.push(`%${query}%`, `%${query}%`);
    }

    const [students] = await pool.query(sql, params);
    res.render('supervisor/_students', { students }); // Частичный шаблон
});

//API для добавление в избранное
app.post('/api/favorites', authenticateToken, async (req, res) => {
    try {
        const { diploma_id, action } = req.body;

        if (action === 'add') {
            await pool.query(
                'INSERT INTO favorites (user_id, diploma_id) VALUES (?, ?)',
                [req.user.id, diploma_id]
            );
        } else {
            await pool.query(

```



```

        'DELETE FROM favorites WHERE user_id = ? AND diploma_id = ?',
        [req.user.id, diploma_id]
    );
}

res.json({ success: true });
} catch (error) {
    console.error('Ошибка обновления избранного:', error);
    res.status(500).json({ error: 'Ошибка сервера' });
}
});

// Старт сервера
app.listen(config.PORT, () => {
    console.log(`Сервер запущен на http://localhost:${config.PORT}`);
});

```